



UNIVERSIDAD DE CHILE
FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
DEPARTAMENTO DE LAS CIENCIAS DE COMPUTACIÓN

IMPLEMENTACIÓN DE UN SERVIDOR DE NOMBRES USANDO LA LIBRERÍA
DNS-RUST

MEMORIA PARA OPTAR AL TÍTULO DE
INGENIERO CIVIL EN COMPUTACIÓN

BASTIÁN NICOLÁS CORRALES CAPDEVILLA

PROFESOR GUÍA:
JAVIER BUSTOS

MIEMBROS DE LA COMISIÓN:
JAVIER BUSTOS
PIQUER
JOSÉ URZÚA

SANTIAGO DE CHILE
2025

Esto va para mi queridísima familia, mis amigos y mis perritos que tanto amo

Agradecimientos

Dedicarle esta memoria a mi familia, a mi mamá, papá, hermano que los amo mucho y que siempre me han apoyado en todo el proceso y la carrera. Agradecer también a mis amigos de la facultad y a los que no son de ahí, ya que siempre han sido un pilar en mi vida, haciendola chistosa, entretenida y acompañando tanto en los momentos malos como los buenos. También agradecer al NIC Labs por aceptarme como uno más del grupo, por ser tan flexibles conmigo y por confiar, especialmente a mi profe guía que nunca me exigió mucho pero siempre confió en que yo podía por mi cuenta y que cada vez que le preguntaba algo por telegram respondía de manera inmediata.

Tabla de Contenido

1. Introducción	1
1.1. Contexto	1
1.2. Glosario	2
1.3. Objetivos	4
2. Marco teórico	5
2.1. RFC's	5
2.1.1. RFC 1034	5
2.1.2. RFC 1035	5
2.1.3. RFC 4033	6
2.1.4. RFC 4034	6
2.2. Estado del Arte	6
2.2.1. ISC BIND server heap overflow de 4 bytes	6
2.2.2. Vulnerabilidades en Dnsmasq	7
2.2.3. Por qué seguir los RFC	8
2.2.4. Otras implementaciones de Name Servers en Rust	8
3. Conexiones UDP y TCP	10
3.1. UDP	10
3.2. TCP	14
4. Estructura y procesamiento de datos	15

4.1.	Masterfile	15
4.2.	Catalogo	16
4.3.	Hash anidado de Resource Records Set (RRset)	17
4.4.	Procesamiento del Masterfile	18
4.4.1.	read_masterfile_records	18
4.4.2.	parse_resource_records	20
4.4.3.	group_rrs_into_rrsets	21
4.5.	Descarte del árbol jerárquico	22
4.5.1.	RFC 1035	22
4.5.2.	Problemas de implementación	22
5.	Algoritmo de Búsqueda	24
5.1.	Flujo y funcionamiento	24
5.1.1.	find_zone_in_catalog	26
5.1.2.	resolve_qname_in_zone	27
5.1.3.	add_glue_records_and_wildcard	29
5.1.4.	add_rrsigs_for_delegation	31
5.1.5.	add_rrsigs_for_answer	31
5.1.6.	add_nsecs_proofs	31
5.2.	Resultados según cada caso de consulta	32
5.2.1.	Consulta con coincidencia exacta con nombre de dominio y tipo . . .	33
5.2.2.	Consulta con coincidencia solo en el nombre de dominio, pero existe un registro CNAME	33
5.2.3.	Consulta que sale de la zona autoritativa : Delgation con Glue Records	34
5.2.4.	Consulta sin coincidencia en el nombre de dominio pero existe un wildcard	35
5.2.5.	Consulta sin coincidencia en nombre de dominio y sin wildcard: NX-DOMAIN	36
5.2.6.	Consulta sin coincidencia en el tipo: NO_DATA	36

6. EDNS0	38
6.1. Características principales de EDNS0	38
6.2. Ejemplo de mensaje DNS con EDNS0	39
7. DNSSEC	40
7.1. Servidor	40
7.1.1. Generación de llaves criptográficas	40
7.1.2. Estructuras de datos para las llaves	42
7.1.3. Inserción de DNSKEY's RRSETS en la estructura de datos	44
7.1.4. Inserción de los RRSIG de los DNSKEY's RRSET	44
7.1.5. Creación e inserción de firmas y el manejo de casos bordes	46
7.1.6. Creación e inserción de los registros NSEC	46
7.1.7. Inserción de RRSIGS para los NSEC RRSET	47
7.1.8. Creación del RRSET DS para la zona padre	48
7.2. Resolver	48
7.2.1. Proceso de autenticación para nombres inexistentes (NXDOMAIN) .	51
7.2.2. Proceso de autenticación para respuestas NO_DATA	52
7.2.3. Autenticación para respuestas provenientes de Wildcards	53
7.2.4. Proceso de Autenticación DNSSEC Estándar	55
7.2.5. Proceso de verificación de la cadena de autenticación	57
7.2.6. Ejemplo real de validación DNSSEC	59
8. Discusiones y Resultados	64
8.1. Uso del lenguaje Rust y sus desventajas	64
8.2. Uso de mapas anidados en lugar de árboles	64
8.3. Limitaciones y errores encontrados en la librería dns-rust	65
8.3.1. Funciones de parsing	65
8.3.2. Falta de implementación de <code>to_canonical_bytes</code> para los Rdata . .	66

8.3.3. Bug en la función de inserción en la sección Authority	66
8.4. Testing del proyecto	67
8.4.1. Servidor UDP con soporte para consultas paralelas	67
8.4.2. Servidor híbrido: manejo de truncamiento (TC) y consultas paralelas en TCP	68
8.4.3. Manejo de consultas mal formadas (FORMERR)	69
8.4.4. Manejo de consultas DNSSEC en UDP y TCP	70
8.4.5. Verificación local de estructuras de datos y RRsets DNSSEC	70
8.4.6. Casos de autenticación criptográfica DNSSEC	72
9. Conclusiones	73
Bibliografía	76

Índice de Tablas

Índice de Ilustraciones

Capítulo 1

Introducción

1.1. Contexto

DNS (Domain Name System) es un protocolo fundamental para el funcionamiento de la capa siete del modelo OSI, permitiendo que un usuario pueda acceder a una IP global a través de un nombre de dominio, como por ejemplo `dcc.uchile.cl`. Para lograr esto existen 2 actores principales: el Resolver y el Name Server.

El Resolver es el programa encargado de tomar una consulta de un nombre de dominio desde un dispositivo como un computador o un celular y transformarlo en una consulta DNS, la cual es un mensaje en bytes con una estructura definida por el protocolo DNS, con las siguientes 5 secciones: Header, Question, Answer, Authority y Additional

El Name Server es el que conoce las IP de los dominios y es el encargado de responder las consultas DNS que envía el Resolver, pero este proceso no es sencillo y requiere del arduo estudio y programación de los RFC (Request For Comments), documentos estandarizados de cómo debe funcionar el internet.

En este contexto, el área de investigación de NIC Chile (NIC Labs) implementó una librería de funciones para DNS en el lenguaje Rust, como caso de prueba de dicha librería desean implementar un servidor de nombres (Name Server) DNS que utilicen para su propia zona `niclabs.cl`. Una zona es un conjunto de nombres de dominios hijos de un dominio padre, en este caso el dominio padre es `niclabs.cl`. (la zona misma) y un dominio hijo podría ser `a.niclabs.cl`

La librería DNS-RUST de NIC Labs sigue cabalmente las indicaciones de los RFC sin implementar ningún tipo de optimización que no sea referenciada por ellos, por lo que es importante la implementación del Name Server bajo esas especificaciones ya que puede ser utilizado más adelante para investigación y desarrollo.

Mencionar que la librería `dns-rust` es solo una herramienta que contiene las funciones necesarias para transformar un mensaje DNS en bytes a una estructura en Rust (parse), un Cliente y un Stub-Resolver, es decir, no posee nada relacionado a un Name Server. De esta

manera todas las funcionalidades, estructuras de datos y algoritmos serán implementados desde cero.

Como se verá más adelante, implementar un Name Server con todas sus funcionalidades incluyendo las que aún no posee la librería es un trabajo que toma más de un semestre, de hecho, para tener más contexto, el Stub-Resolver de la librería DNS-RUST fue desarrollado por años de manera grupal, por lo que es lógico pensar que su contraparte (un Name Server) no puede ser completado al cien por ciento por una sola persona en un semestre, provocando así que el trabajo a realizar sea acotado sólo a las funcionalidades de DNS Security Extension, Extension DNS 0, creación de estructuras de datos basadas en el `masterfile`, implementación del algoritmo de búsqueda del Name Server y conexiones UDP-TCP asíncronas.

1.2. Glosario

Antes de continuar con el desarrollo de este informe, es necesario abarcar ciertos conceptos fundamentales que se utilizarán en las secciones posteriores:

- **Stub-Resolver:** Resolver simple que delega completamente el trabajo a un resolver recursivo externo. Estos resolvers recursivos suelen ser complejos y, en la práctica, un stub-resolver utiliza servicios de grandes proveedores como Google o Cloudflare.
- **Extension DNS 0 (EDNS0):** Primera gran extensión del protocolo DNS. Añade nuevas funcionalidades, entre ellas la posibilidad de ampliar el tamaño máximo de los mensajes DNS (consultas y respuestas), superando el límite tradicional de 512 bytes.
- **DNS Security Extension (DNSSEC):** Extensión de seguridad del protocolo DNS que permite incorporar firmas digitales y realizar validación criptográfica de la información contenida en los Resource Records.
- **Resource Record (RR):** Unidad básica de información en DNS. Cada RR representa un dato asociado a un nombre de dominio y tiene un tipo específico con su propia estructura.
- **RRset:** Conjunto de RR que comparten el mismo nombre de dominio, clase y tipo.
- **Catálogo:** Estructura jerárquica que contiene las zonas administradas por el Name Server.
- **Árbol de dominios:** Estructura jerárquica que almacena los RRsets asociados a los dominios que maneja el Name Server.
- **Tipos de Resource Records clásicos:**
 - **Answer (A):** Contiene la dirección IPv4 asociada al nombre de dominio solicitado.
 - **Name Server (NS):** Indica el nombre de dominio del servidor autoritativo responsable de una zona.

- **Canonical Name (CNAME)**: Alias que redirige un nombre de dominio a otro nombre canónico.
 - **SOA (Start of Authority)**: Contiene información administrativa sobre la zona, como el servidor principal, el correo del administrador, números de serie y tiempos de actualización.
 - **Optional (OPT)**: RR especial introducido por EDNS0. Su función más relevante es permitir aumentar el tamaño máximo de los mensajes DNS más allá del límite clásico de 512 bytes.
- **Tipos de Resource Records de DNSSEC**:
 - **DNS Public Key Record (DNSKEY)**: Contiene la clave pública utilizada para validar firmas digitales en un RRSIG.
 - **Resource Record Signature (RRSIG)**: Firma criptográfica generada con la clave privada del servidor autoritativo, y que autentica un RRset completo.
 - **Delegation Signer (DS)**: Hash de la clave pública contenida en un DNSKEY, almacenado en la zona padre para validar la delegación.
 - **Next Secure Record (NSEC)**: Indica el siguiente nombre válido en el orden canónico dentro de la zona y enumera qué tipos de RR existen para un dominio. Se utiliza en pruebas de no existencia.
 - **Clase de un RR**: Valor que determina el contexto o protocolo asociado. Normalmente es IN (Internet), aunque existen otras clases menos utilizadas.
 - **RDATA (Response Data)**: Parte del RR que contiene la información específica del tipo correspondiente. Su estructura depende del tipo de RR.
 - **Glue Record**: RR especial que contiene la dirección IP de un Name Server delegado en otra zona, utilizado para evitar ciclos infinitos durante la resolución.
 - **Wildcard**: Nombre de dominio generalizado que permite responder por múltiples subdominios ausentes, utilizando el símbolo asterisco (*). Por ejemplo, `*.niclabs.cl` puede responder consultas como `a.niclabs.cl` o `b.c.niclabs.cl`.
 - **NXDOMAIN**: Código de error que indica que el nombre de dominio consultado no existe en la zona.
 - **NO_DATA**: Código de error que indica que el nombre de dominio consultado sí existe, pero no posee registros del tipo solicitado.
 - **Formato canónico**: Representación estandarizada de los datos de un RRset utilizada por DNSSEC para firmar y verificar información. Consiste en ordenar lexicográficamente los registros y normalizar cada campo, garantizando que la firma criptográfica sea siempre reproducible e independiente del orden original de los datos.

1.3. Objetivos

Objetivo General

El objetivo de este trabajo de título es construir un MVP (Producto Mínimo Viable) de un Name Server Autoritativo Iterativo capaz de controlar la zona de dominio del laboratorio de manera segura, eficiente, siguiendo al pie de la letra los RFC y usando las herramientas de la librería DNS-RUST como apoyo.

Que el Name Server sea Iterativo consiste en que su algoritmo busca el nombre de dominio solicitado dentro de su estructura de datos, si no tiene la respuesta directa, envía una referencia hacia otro Name Server que si podría tenerla, y si no encuentra ninguna respuesta envía un error. La alternativa es un Name Server recursivo que, si no conoce la respuesta a la consulta, utiliza un Resolver interno para averiguarla y responder, y que por su diseño y construcción no es parte de este trabajo de título.

Objetivos Específicos

1. Establecer canales de transmisión confiables, en los protocolos de UDP y TCP asincrónicos.
2. Responder correctamente a consultas DNS de tipo basico (A, NS, CN) con sus Resource Records correspondientes.
3. Seguir estrictamente lo establecido por los RFC antiguos y actuales.
4. Asegurar seguridad con DNSSEC (DNS Security Extension), usando algoritmos de encriptación, llaves publicas y privadas, firmas y una cadena de autenticación.
5. Crear de estructuras de datos a partir de un `masterfile` .
6. Conectar las herramientas de la libería `dns-rust` usando su stub-resolver como cliente y sus parses de mensajes para manejar las consultas DNS .
7. Cumplir con estándares de documentación en el código.
8. Incluir EDNS0 (Extension DNS 0) necesaria para cumplir DNSSEC. Especificamente sabiendo leer y procesar consultas del tipo OPT.

Capítulo 2

Marco teórico

2.1. RFC's

Parte fundamental del protocolo DNS son los RFC, ya que definen las reglas y el comportamiento que debe seguir todo el internet en general, y en específico el protocolo DNS. A continuación se detallan los RFC más importantes para el desarrollo de este proyecto:

2.1.1. RFC 1034

- Estructura de una consulta DNS, en específico, el formato de la sección Header.
- Resource Records, explica campos como el owner name, tipo, clase, TTL (Time To Live) y RDATA (Response Data)
- Algoritmo de búsqueda de un Name Server para un nombre de dominio.
- Algoritmo de consulta de un Resolver.
- WildCards, nombres de dominios especiales que permiten que subdominios no definidos respondan con la misma información.

2.1.2. RFC 1035

- Significado de los valores y flags dentro del Header.
- Formatos de los RDATA de tipos comunes de RR (A, SOA, NS, CNAME).
- Conexiones UDP y TCP y sus usos en cada caso.
- `masterfiles` y como a partir de estos crear las estructuras de datos.
- Arquitectura del Name Server, específicamente el uso de la concurrencia-asincronismo en el procesamiento de consultas.

2.1.3. RFC 4033

- Glosario de conceptos DNSKEY: Cadena de autenticación, pares de llaves criptográficas, puntos de delegación, etc.
- Dos nuevos bits en el Header: Checking Disabled (CD) y Authenticated Data (AD).
- Uso de EDNS0 para aumentar el tamaño base (512 bytes) de una consulta DNS. Esto ya que las llaves y firmas usadas en DNSSEC suelen ser de gran tamaño.

2.1.4. RFC 4034

- Formatos de los RDATA de los tipos de DNSSEC (DNSKEY, RRSIG, DS y NSEC)
- Forma Canonica para ordenar los RRSets y los nombres de dominios.
- Algoritmos de encriptación
- Algoritmo para calcular el Key Tag del RRset de DNSKEY

2.2. Estado del Arte

En el contexto actual existen distintos software para crear un Name Server, algunos comerciales como Bind9 [10] que está escrito en C y que es Open Source y otros como PowerDNS [19] que es de código privado. Estas son herramientas útiles, pero no es posible verificar si cumplen estrictamente con las especificaciones de los RFC, lo que resulta especialmente importante para NIC Labs. Además, estos programas no están escritos en Rust, lenguaje de programación muy valorado por su alto nivel de seguridad en accesos a memoria [9].

Es importante tener seguridad en el acceso a memoria, ya que previamente se han encontrado o aprovechado vulnerabilidades de Name Servers escritos en otros lenguajes. A continuación se verán algunos ejemplos:

2.2.1. ISC BIND server heap overflow de 4 bytes

Como se vio previamente Bind9 es uno de los software más usados para crear Name Servers, pero a pesar de esto en febrero de 2021 se reportó y se parchó una vulnerabilidad que se mantuvo por 15 años sin ser explotada, y que fue descubierta por un investigador anónimo [24].

La vulnerabilidad permite de manera remota y sin autenticación envenenar el caché del server (Poisoned Cache Attack), provocando así que el caché guarde y responda IPs a sitios inseguros a consultas legítimas, afectando gravemente a todos los usuarios que consulten a este Name Server.

La vulnerabilidad nace por el siguiente código escrito en C:

```

1 static int
2 der_get_oid(const unsigned char *p, size_t len, oid *data, size_t *size) {
3 // ...
4 data->components = malloc(len * sizeof(*data->components)); // <-- (1)
5 if (data->components == NULL) {
6     return (ENOMEM);
7 }
8 data->components[0] = (*p) / 40; // <-- (2)
9 data->components[1] = (*p) % 40;
10 --len; // <-- (3)
11 ++p;
12 for (n = 2; len > 0U; ++n) {
13     unsigned u = 0;
14
15     do {
16         --len;
17         u = u * 128 + (*p++ % 128);
18     } while (len > 0U && p[-1] & 0x80);
19     data->components[n] = u; // <-- (4)
20 }
21 // ...
22 }

```

La función provoca un overflow de 4 bytes al final del arreglo, permitiendo al atacante utilizar esos 4 bytes para inyectar código malicioso y provocar el envenenamiento del caché.

Con el lenguaje Rust habría sido más difícil que ocurriera, ya que el compilador hubiese prevenido al programador al momento de asignar un valor fuera de los límites del vector original.

2.2.2. Vulnerabilidades en Dnsmasq

Dnsmasq [11] es un redireccionador de DNS usado por servidores que en primera instancia no quieren usar la red global de Name Servers y utilizar únicamente el caché proporcionado por este servicio. En general se usa para sistemas embebidos con zonas muy pequeñas que no necesitan mucha comunicación con otros servidores.

El problema con este software escrito en el lenguaje C es que presentó múltiples vulnerabilidades simultáneamente, en concreto se encontraron 4 vulnerabilidades de buffer overflow (provocadas por errores directos en el código) y 3 vulnerabilidades de validación que podrían provocar envenenamiento del caché, a continuación se muestran las últimas 3 vulnerabilidades:

- CVE-2020-25681: No se validaba el par (dirección, puerto) ni el ID de la consulta DNS al recibirlas.
- CVE-2020-25685: Usaba un algoritmo de hashing débil (CRC32), vulnerable a ataques.
- CVE-2020-25686: Dnsmasq no chequeaba que una consulta hubiese terminado de procesarse para recibir y redireccionar otra igual (con el mismo nombre de dominio, tipo y

clase), por lo que el atacante puede enviar muchas consultas equivalentes tratando de adivinar el ID para que así el cliente la acepte como válida, y esto no es tan improbable por la paradoja del cumpleaños [7]

A pesar de que estas 3 vulnerabilidades mencionadas no sean consecuencia directa del lenguaje usado, sino de la mala verificación y uso de algoritmos, es fundamental conocerlas y analizarlas para evitar repetir errores ya cometidos.

2.2.3. Por qué seguir los RFC

Los RFC son los documentos estandarizados que explican cómo debe ser implementado el internet en sus diferentes capas, pero, ¿Qué consecuencias puede tener ignorar las reglas de estos documentos?

A continuación se verán vulnerabilidades descubiertas en Name Servers que no siguieron indicaciones **obligatorias** dadas por los RFC.

Falta de validaciones en los nombres de dominio

Según el RFC 1035 [15] los nombres de dominio deben seguir un estándar y cumplir ciertas reglas, en específico deben cumplir lo siguiente:

- Tener un largo de máximo 255 bytes en total.
- Cada label dentro del nombre de dominio puede tener un máximo de 63 bytes.
- Terminar con un carácter \0.

Todas esas condiciones deben ser verificadas, de lo contrario pueden ocurrir diversos ataques de buffer overflow, ataques de Denegación de Servicio (DDoS) y ejecución de código remoto [8].

En concreto se descubrió una vulnerabilidad [12] consecuencia de la falta de validación del largo de un nombre de dominio dentro de la implementación de DNS en Nut/OS en un dispositivo Ethernut (dispositivo de IoT). Solamente con esta falla se podría ejecutar código remoto y/o un ataque DDoS.

De manera similar, se encontraron muchas vulnerabilidades en dispositivos IoT causadas por la falta de validación del largo de los labels del nombre de dominio [13].

2.2.4. Otras implementaciones de Name Servers en Rust

Existen otras librerías de DNS (en específico con el módulo de Name Server) que están siendo desarrolladas por otras organizaciones, como **HickoryDNS** [6] y **domain**[18] de NL-

netLabs. Estas librerías están en un punto muy alto de desarrollo, pero es importante conocer y comprender cuáles han sido sus errores. A continuación se verán algunos ejemplos:

Hickory vulnerabilidad DNSSEC por verificación de un solo DNSKEY

En el Resolver de Hickory al momento de validar un DNSKEY a través del ancla de confianza (trust anchor), que es un registro ya guardado en el Resolver para terminar la cadena de autenticación, si el DNSKEY es validado, por error se valida cualquier otro DNSKEY proveniente del mismo RRSset [5], lo cual es incorrecto, ya que cada uno de estos debe ser validado de forma individual.

Error al firmar la zona debido a glue records

Para firmar una zona (aplicando DNSSEC) se necesita encontrar el RR **SOA**, el cual permite verificar que la zona es legítima, y este RR siempre debe estar en el Apex Zone (en la zona más alta de la estructura de datos jerárquica), pero si hay algún glue RR (los cuales tienen las IP de otros Name Servers) estos pueden anteceder al **SOA** debido al ordenamiento canónico (por bytes).

Este reordenamiento provocaba que el proceso de firma no encontrara el **SOA** RR y por lo tanto no firmara una zona válida [17].

Capítulo 3

Conexiones UDP y TCP

El servidor DNS implementado usa la librería `tokio` para atender consultas en UDP y TCP de forma asíncrona y concurrente. Además de la funcionalidad básica de recibir y devolver respuestas, también soporta EDNS0, detecta y respeta el payload (tamaño del mensaje) máximo anunciado por el cliente en el registro `OPT`, maneja el estándar DNSSEC (lectura del bit `DO`) y protege operaciones críticas con timeouts para evitar bloqueos y bucles infinitos. También se manejan correctamente errores de parseo (se responde `FORMERR`) y se implementa el comportamiento para truncamiento (TC) sobre UDP con fallback a TCP.

A continuación se describen las implementaciones específicas para cada protocolo.

3.1. UDP

El servidor UDP está implementado en la estructura `ServerUDPConnection`, que contiene los siguientes campos y métodos:

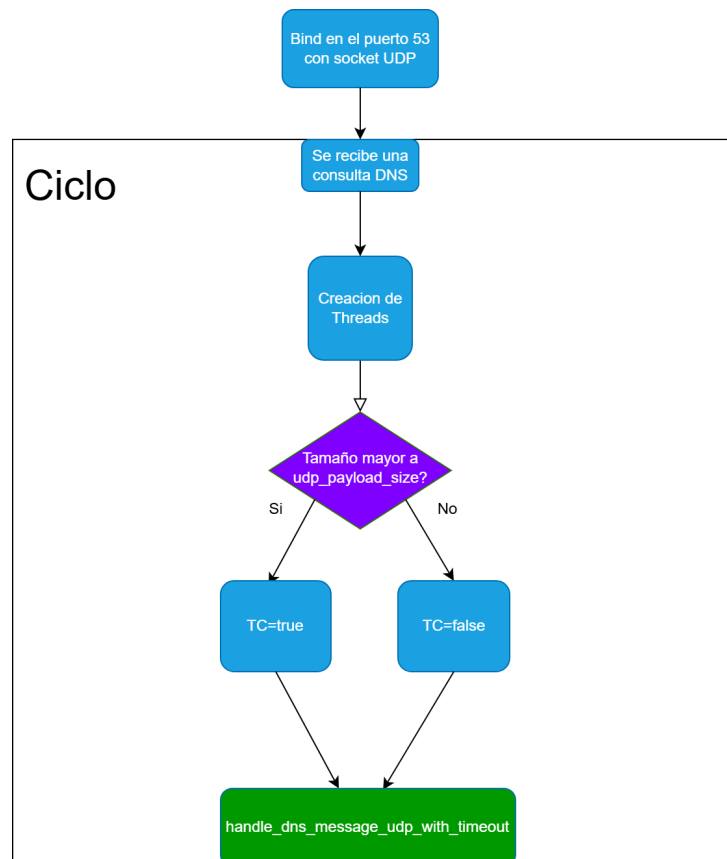
ServerUDPConnection
Campos
- <code>timeout: tokio::time::Duration</code> - <code>payload_size: usize</code>
Métodos Públicos
+ <code>new_udp(timeout: Duration, payload_size: usize) -> Self</code> + <code>new_default(timeout: Duration) -> Self</code> + <code>receive_and_response(&mut self) -> Result<(), ServerError></code>
Métodos Privados / Auxiliares
- <code>handle_dns_message_udp_with_timeout (</code> <code>timeout_server: Duration, recv_buf: &[u8],</code> <code>socket: &UdpSocket, client_addr: SocketAddr,</code> <code>tc_case: bool, len: usize) -> Result<(), ServerError></code>

El campo `timeout` define el tiempo máximo que el servidor esperará para recibir una consulta o enviar una respuesta antes de abortar la operación. El campo `payload_size` establece el tamaño máximo de la consulta DNS que el servidor puede procesar.

Los métodos definidos incluyen funciones típicas como `new` y `new_default`, que permiten crear una instancia de la clase con parámetros personalizados o por defecto. Sin embargo, la función más relevante es `receive_and_response`, la cual se encarga de recibir y responder de manera asíncrona a las consultas DNS enviadas por los clientes. Además esta función tiene en cuenta el tamaño máximo que puede soportar el servidor al momento de recibir consultas UDP, ya que si el tamaño de la consulta excede el `payload_size` se activa el bit TC en la respuesta para que el cliente reintente la consulta por TCP.

La función `receive_and_response` sigue los siguientes pasos:

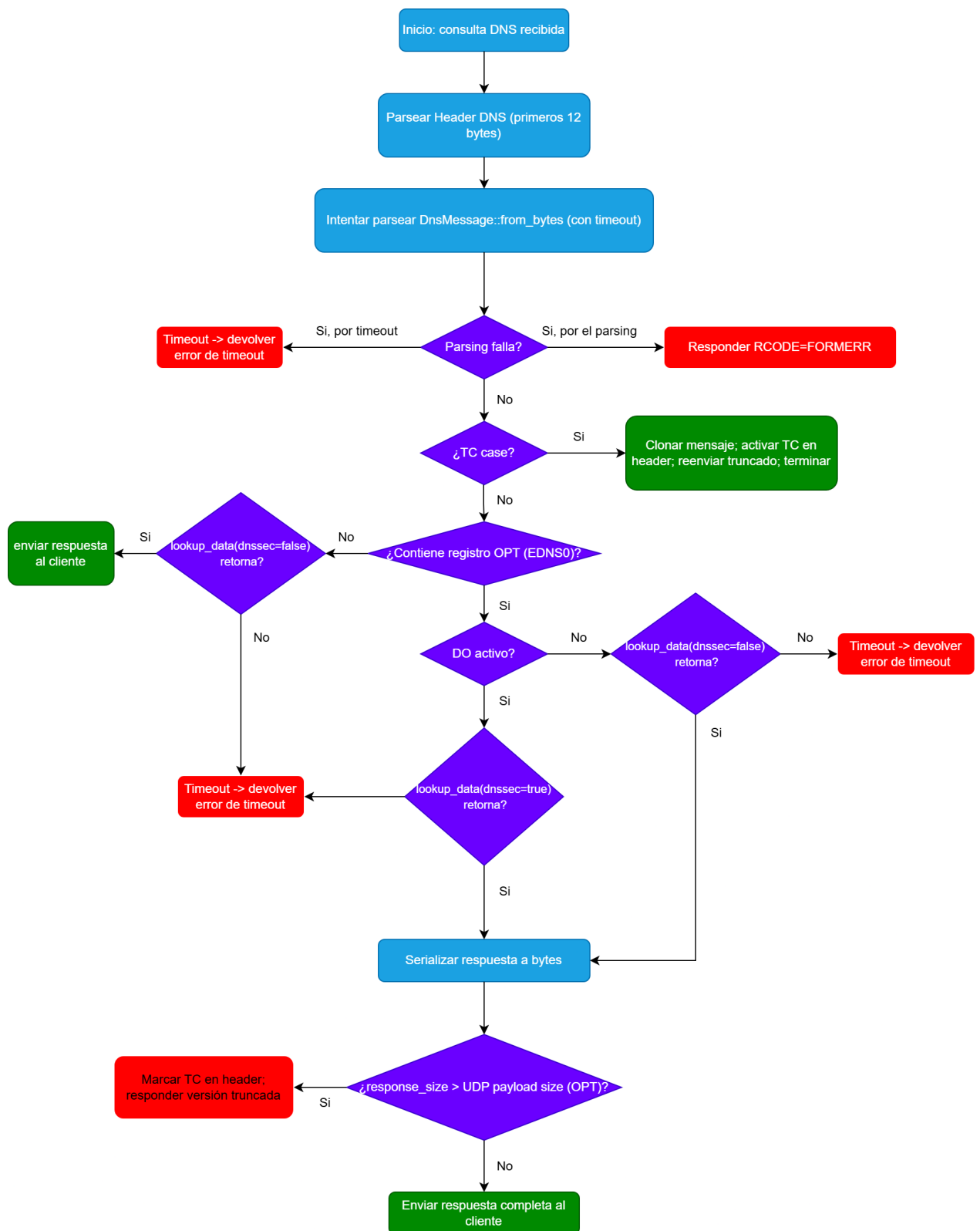
- Crea un socket UDP y se enlaza a localhost en el puerto 53.
- Entra en un bucle infinito donde se espera recibir consultas DNS.
- Crea threads con `tokio` para manejar cada consulta de forma concurrente.
- En cada thread, si el tamaño de la consulta es mayor al `payload_size`, se usa la función `handle_dns_message_udp_with_timeout` con `tc_case` en verdadero, de lo contrario se llama a la misma función con `tc_case` en false. `tc_case` indica si se debe manejar el caso de truncamiento (TC).



Con respecto a la función auxiliar `handle_dns_message_udp_with_timeout`, esta se encarga de:

- Parsear el **Header DNS**, correspondiente a los primeros 12 bytes del buffer recibido.
- Intenta parsear el **mensaje DNS completo** bajo un `timeout`.
 - Si falla el parsing, responde con un `RCODE=FORMERR`.
 - Si expira el `timeout`, devuelve un error de `timeout`.
- Si `tc_case` es verdadero:
 - Clona el mensaje original.
 - Activa el bit **TC** en el header de respuesta.
 - Reenvía el mensaje original truncado al cliente.
 - Termina la ejecución.
- Si no es un TC case, busca si el mensaje contiene un registro **OPT** (EDNS0):
 - Si hay algún **OPT**, obtiene el **DO bit** desde el campo `TTL`.
 - Dependiendo del **DO bit**:
 - Si está activo, llama a `lookup_data` con `dnssec=true`.
 - Si no está activo, llama a `lookup_data` con `dnssec=false`.
 - Transforma la respuesta en bytes.
 - Compara el tamaño de la respuesta con el valor de `UDP payload size` del registro **OPT**.
 - Si la respuesta es mayor al tamaño permitido por el cliente:
 - Marca el **TC bit** en la cabecera original.
 - Responde con la versión truncada.
 - Si cabe dentro del tamaño permitido:
 - Envía la respuesta completa.
- Si no hay EDNS0 (sin registro **OPT**):
 - Llama a `lookup_data` con `dnssec=false`.
 - Envía la respuesta al cliente.

El diagrama de flujo de la función `handle_dns_message_udp_with_timeout` se muestra a continuación:



3.2. TCP

El servidor TCP está implementado en la estructura `ServerTCPConnection`, que contiene los siguientes campos y métodos:

ServerTCPConnection	
Campos	
- <code>timeout: tokio::time::Duration</code> - <code>payload_size: usize</code>	
Métodos Públicos	
+ <code>new(timeout: Duration, payload_size: usize) -> Self</code> + <code>new_default(timeout: Duration) -> Self</code> + <code>get_timeout(self: Self) -> tokio::time::Duration</code> + <code>receive_and_response(&self) -> Result<(), ServerError></code>	
Métodos Privados / Auxiliares	
- <code>handle_dns_message_with_timeout(timeout_server: Duration, recv_buf: &[u8], socket: &mut TcpStream) -> Result<(), ServerError></code> - <code>handle_connection(timeout_server: Duration, socket: TcpStream, addr: std::net::SocketAddr) -> Result<(), ServerError></code>	

La versión TCP del servidor comparte gran parte de su estructura con el modo UDP, ya que utiliza los mismos campos principales y métodos similares para la configuración y el procesamiento de consultas.

La principal diferencia radica en la función `handle_connection`, la cual se encarga de gestionar cada conexión TCP entrante. Esta función primero lee un prefijo de 2 bytes que indica la longitud total del mensaje DNS (requisito del protocolo cuando se utiliza TCP) y, a continuación, recibe el mensaje completo. Una vez obtenido, delega su procesamiento a la función `handle_dns_message_with_timeout`.

En este caso la función `handle_dns_message_with_timeout` no necesita manejar el caso de truncamiento (TC), ya que TCP garantiza la entrega completa del mensaje. Además es prácticamente idéntica a la vista en UDP pero con la diferencia que antes de mandar el mensaje final (sea el caso que sea) debe anteponer el prefijo de 2 bytes con la longitud del mensaje a enviar.

Capítulo 4

Estructura y procesamiento de datos

4.1. Masterfile

Un masterfile es un archivo de texto que contiene los registros DNS de una zona en un formato estandarizado. Se trata de la representación más básica y tradicional para almacenar la información de una zona. Cada línea del archivo corresponde a un Resource Record, cuyos campos están separados por espacios o tabulaciones. Además, el formato admite directivas especiales que permiten definir parámetros globales para la zona, como el valor predeterminado del TTL (Time To Live) o el servidor de nombres autoritativo.

A continuación se muestra un ejemplo con datos ficticios para la zona example.com:

```
1 example.com.      IN SOA ns1.example.com. hostmaster.example.com. (  
2                   2025052101 ; serial  
3                   3600      ; refresh  
4                   1800      ; retry  
5                   1209600    ; expire  
6                   3600      ; minimum TTL  
7                   )  
8 example.com.      IN NS  ns1.example.com.  
9 example.com.      IN NS  ns2.example.com.  
10 *.example.com.    IN A  0.0.0.0  
11  
12 www.example.com.  IN A    1.2.3.4  
13 www.example.com.  IN A    1.1.1.2 ; second IP  
14 mail.example.com. IN A    2.2.2.2  
15 blog.example.com. IN CNAME blog2.example.com.  
16 blog2.example.com. IN A    1.2.3.4  
17 ns1.example.com.  IN A    3.3.3.3  
18  
19 b.example.com.     IN NS  ns.a.b.example.com. ; delegation point  
20 ns.a.b.example.com. IN A  4.3.2.1 ; name server address (glue)
```

Listing 4.1: Contenido del masterfile de la zona example.com

Como se aprecia en el ejemplo, es posible incluir comentarios utilizando el símbolo `;`; asimismo, cuando un registro es extenso, se permite dividirlo en múltiples líneas encerrando su contenido entre paréntesis `()`, lo que mejora significativamente la legibilidad. Incluso dentro de estos paréntesis se pueden insertar comentarios adicionales.

En resumen, un masterfile almacena la información en texto plano, de manera sencilla y legible para humanos. Sin embargo, este formato no es eficiente para responder consultas DNS directamente, por lo que es necesario procesarlo y transformarlo en estructuras de datos internas más adecuadas. A continuación, se describe cómo se realiza este procesamiento y las estructuras utilizadas para representar la información de la zona en memoria.

4.2. Catalogo

Estructura que contiene un `HashMap` donde cada llave es un string con el nombre de la zona y el valor es un puntero a la estructura con los datos reales. Es básicamente un índice que permite acceder rápidamente a los datos de cada zona, y es especialmente útil cuando el servidor maneja múltiples zonas DNS. También es útil para saber si una consulta pertenece a una zona manejada por el servidor o no, ya que si el dominio consultado no es parte de ninguna zona del catalogo, entonces se puede responder inmediatamente con un `REFUSED`.

El valor de retorno en el `HashMap` está envuelto en un `Arc<RwLock<>>` para permitir el acceso concurrente a los datos de la zona desde múltiples threads.

A continuación se muestra la definición de la estructura `Catalog`:

```
1 pub struct Catalog {  
2     zones: HashMap<String, Arc<RwLock<DataStructure>>>,  
3 }
```

4.3. Hash anidado de Resource Records Set (RRset)

Estructura de datos global accesible por el algoritmo de búsqueda para la obtención de los `RRSet`. En un principio la estructura de datos iba a ser un árbol n-ario jerárquico, pero al momento de implementarlo hubieron muchos problemas, esto se verá con más detalle en la sección 4.5

Esta estructura contiene el nombre de la zona (por ejemplo `niclabs.cl`) y todos los datos asociados organizados en un `HashMap` anidado. La primera llave corresponde al **nombre de dominio**, y la segunda al **Rrtype** (tipo de registro). Esta organización responde al hecho de que el algoritmo de búsqueda debe obtener un `RRset`, el cual está definido como el conjunto de todos los registros (`RR`) que comparten el mismo nombre de dominio, tipo (`Rrtype`) y clase. En esta implementación se asume que la clase es siempre `IN` (Internet), por lo que no se incluye como parte de las llaves del hashmap.

A continuación se muestra la definición de la estructura `DataStructure`:

```
1 pub struct DataStructure {  
2     name: String,  
3     data: HashMap<DomainName, HashMap<Rrtype, RRset>>,  
4 }
```

Aclarar también que para que la estructura sea accesible de manera global y concurrente, se crea la siguiente variable global:

```
1 pub static DATA_STRUCTURE_NICLABS: Lazy<Arc<RwLock<DataStructure>>> =  
    Lazy::new(|| {  
2     let path = 'src/masterfiles/masterfile.txt';  
3     Arc::new(RwLock::new(data_initialization(path, 'niclabs.cl.').  
        unwrap()))  
4 });
```

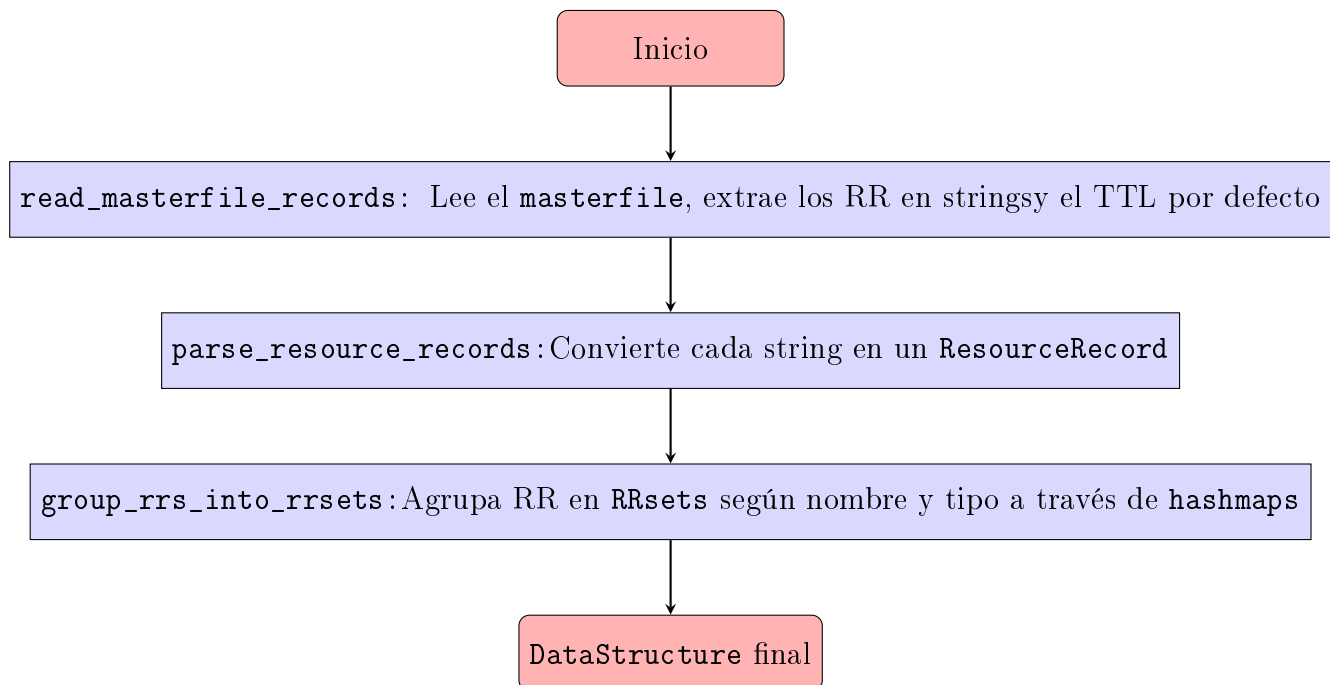
Mencionar que esta variable está envuelta en muchas estructuras, a continuación se explica el uso de cada una:

- **RwLock**: Permite múltiples accesos concurrentes de solo lectura desde distintos hilos (*threads*). Sin embargo, cuando un hilo adquiere acceso de escritura, bloquea todas las lecturas hasta que finalice la operación de escritura.
- **Arc**: Es un puntero inteligente de conteo de referencias atómico que permite compartir datos entre múltiples hilos de manera segura. Es necesario usar un puntero ya que la estructura `Catálogo` debe mantener una referencia compartida a esta información.
- **Lazy**: Es una construcción de evaluación diferida que permite inicializar un valor solo la primera vez que se accede a él. Esto es útil para inicializar estructuras pesadas de forma diferida y segura en tiempo de ejecución.

En primera instancia esa variable global más la misma pero para la zona `example.com` son suficientes, pero como el server emulará zonas padres para lograr la resolución de la cadena de autenticación de DNSSEC, se deberán crear variables globales similares para las zonas `com.`, `cl.` y `root.`.

4.4. Procesamiento del Masterfile

Pasar de la estructura vista en el Masterfile al HashMap anidado de RRset requiere de un proceso extenso de parsing, el cual se puede resumir con el siguiente diagrama:

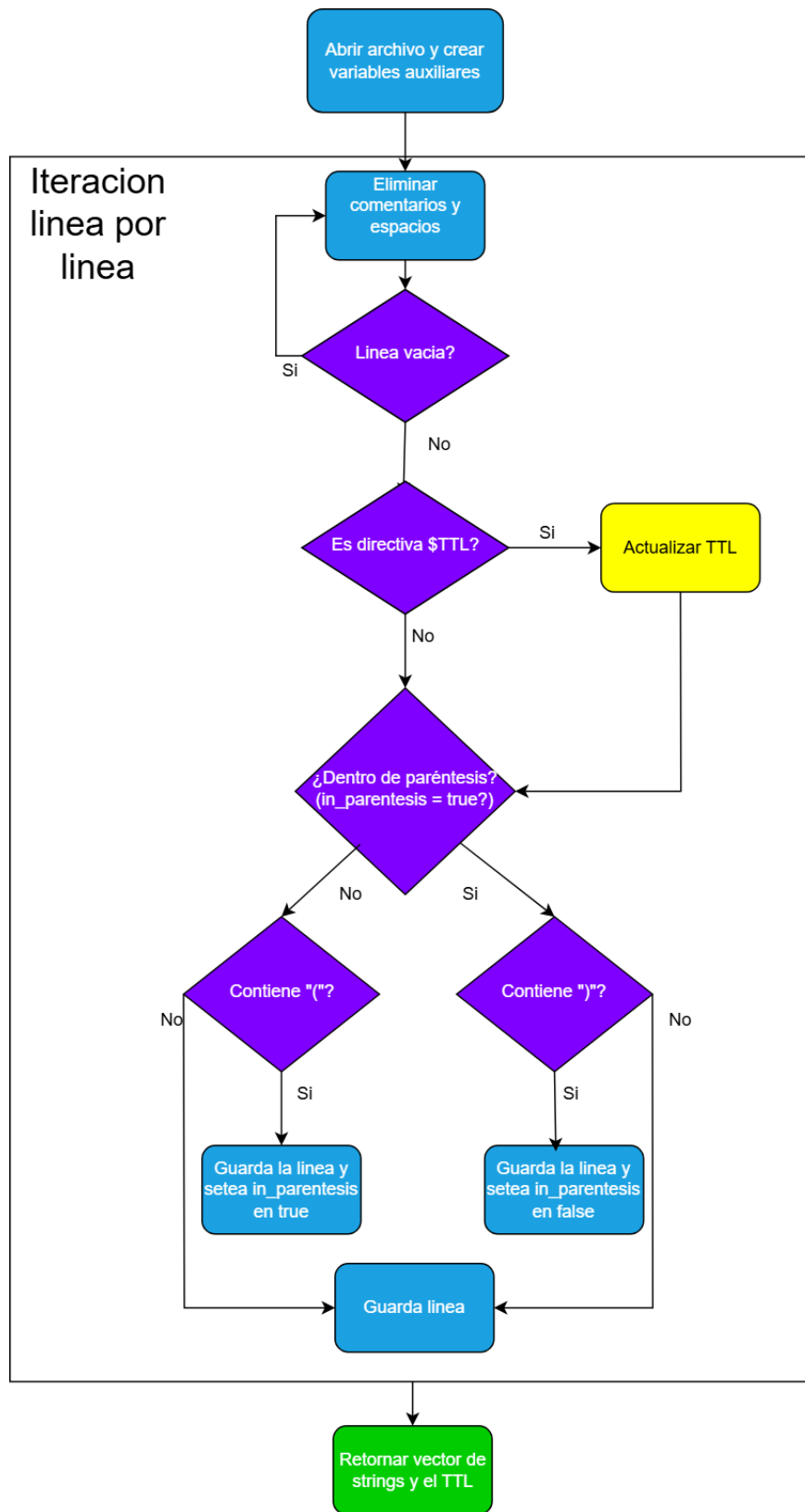


A continuación se describen en detalle cada una de las funciones involucradas en el proceso:

4.4.1. read_masterfile_records

Esta función se encarga de leer el archivo masterfile línea por línea, ignorando comentarios, líneas vacías, manejando los parentesis para registros largos y extrayendo el TTL por defecto si está definido. Cada línea que corresponde a un Resource Record se almacena en un vector de strings, el cual es retornado junto con el TTL por defecto.

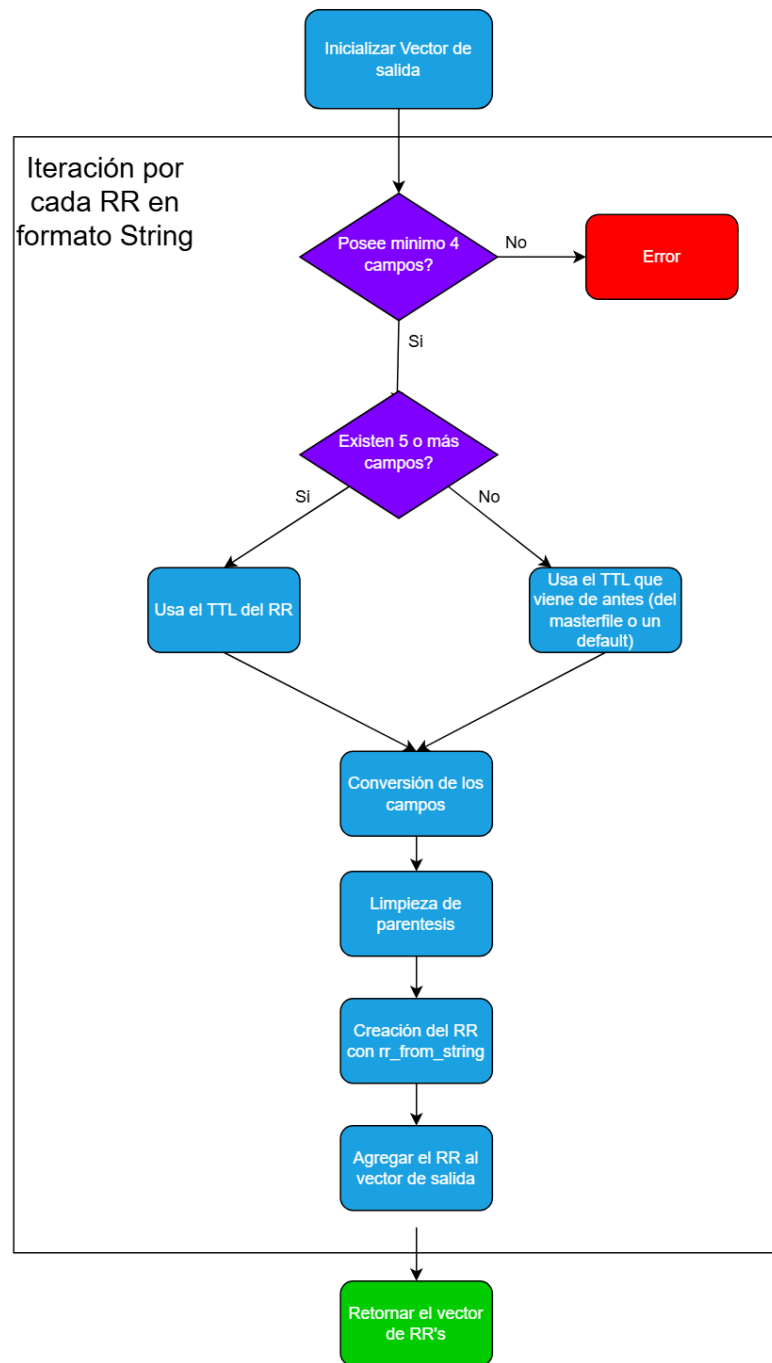
El diagrama de flujo es el siguiente:



4.4.2. parse_resource_records

Esta función recibe el vector de strings generado en la función anterior y convierte cada string en una estructura `ResourceRecord`. Para ello, divide cada línea en sus componentes (nombre, TTL, clase, tipo y datos) y los asigna a los campos correspondientes de la estructura. Si el TTL no está especificado en el registro, utiliza el TTL por defecto extraído previamente.

Su diagrama de flujo es el siguiente:



Como se ve en el diagrama, esta función llama a otra función auxiliar llamada `rr_from_string`, la cual se encarga de convertir un string específico en un `ResourceRecord`. Basicamente esta función hace un `match` y según el tipo del `RR`, llama recursivamente a una función llamada `rr_from_master_file` para cada tipo de `RR`. Esto se hizo así ya que en la librería `dns-rust` ya existía ésta función para la gran mayoría de casos, pero faltaba la implementación para los siguientes dos tipos:

- AAAA
- DS

Estos casos fueron implementados manualmente siguiendo la misma lógica que las funciones ya existentes en la librería.

Aclarar también que no se muestra cada función especializada para cada tipo de `RR` por cuestiones de espacio, pero todas siguen la misma lógica: reciben el nombre, TTL, clase y el iterador de datos, y retornan un `ResourceRecord` con los campos correspondientes llenados.

4.4.3. `group_rrs_into_rrsets`

Esta función recibe el vector de `ResourceRecord` generado previamente y los agrupa en conjuntos `RRset` utilizando una estructura de varios `HashMap` anidados. En primer lugar, la función recorre todos los registros y obtiene, para cada uno, su owner name y su tipo (`Rrtype`). Con ambos valores se construye una llave compuesta que se usa para poblar un primer `HashMap`. Este mapa inicial permite agrupar los registros de acuerdo con la definición de un `RRset` (mismo nombre y mismo tipo), aunque su estructura no es todavía la más conveniente. Por ello, en un segundo paso, se recorre nuevamente este mapa y se genera una nueva estructura jerárquica: el primer nivel corresponde al owner name, el segundo al `Rrtype`, y finalmente, como valor asociado, se almacena un `RRset`. En esta etapa, cada vector de `ResourceRecord` se transforma explícitamente en un objeto `RRset`, ya que, estrictamente hablando, un `RRset` no es simplemente una colección de `RR's`, sino una estructura que encapsula dichos registros junto con metadatos adicionales asociados.

La definición de `RRset` de la librería `dns-rust` es la siguiente:

```
1 pub struct RRset {  
2     name: DomainName,  
3     rrtype: Rrtype,  
4     rclass: Rclass,  
5     ttl: u32,  
6     records: Vec<(u16, Rdata)> //Pair rdlen, rdata  
7 }
```

Como se ve en la estructura, un `RRset` contiene los valores que definen el conjunto (nombre, tipo, clase), un TTL común que suele ser el mínimo de todos los `RR's` y finalmente un

vector con tuplas que contienen la longitud del `Rdata` y el `Rdata` en sí. ¿Por qué se guarda el `Rdata` en vez del `ResourceRecord` completo? Porque sería redundante, ya que los valores comunes a todos los `RR`'s del conjunto ya están almacenados en los campos superiores del `RRset`.

4.5. Descarte del árbol jerárquico

Como se vió en la sección 4.3 se intentó implementar un árbol n-ario jerárquico para almacenar los datos de la zona DNS. Sin embargo, esta implementación presentó múltiples dificultades y problemas que finalmente llevaron a descartarla en favor de una estructura basada en `HashMap` anidados. Primero se explicará que RFC recomienda esta estructura y luego se detallarán los problemas encontrados.

4.5.1. RFC 1035

El RFC 1035 [15], que define el estándar para el sistema de nombres de dominio, **sugiere** que los servidores DNS deberían utilizar una estructura de datos jerárquica para almacenar los registros de recursos (`RR`) de una zona DNS, pero también dice explícitamente que el manejo de estructuras internas es completamente libre. Citando textualmente la sección 6.1.2 del RFC 1035:

“While name server implementations are **free** to use any internal data structures they choose, the suggested structure consists of three major parts:

De esta manera no se está rompiendo con ninguna norma o estándar al optar por una estructura diferente a la sugerida en el RFC.

4.5.2. Problemas de implementación

Implementar una estructura de árbol n-ario jerárquico para almacenar los registros DNS presentó varios desafíos técnicos y conceptuales que dificultaron su desarrollo y finalmente llevaron a su descarte. Algunos de los problemas más relevantes fueron:

- **Complejidad innata de la estructura:** La idea de un árbol es que vaya desde el dominio más general (raíz) hasta el más específico (hojas), pero para lograr esto desde el masterfile se debía agrupar los registros por niveles, lo que implicaba múltiples pasadas sobre los datos y una lógica compleja para manejar casos especiales como `CNAMEs`, delegaciones, etc.

- **Ownership de Rust:** Rust tiene un sistema de ownership muy estricto que complica la implementación de estructuras de datos recursivas como árboles. Manejar referencias mutables y compartidas en un árbol n-ario resultó ser particularmente desafiante, especialmente cuando se necesitaba modificar nodos en diferentes niveles del árbol. Además el sistema debía funcionar en un entorno concurrente, lo que añadió más restricciones.

Capítulo 5

Algoritmo de Búsqueda

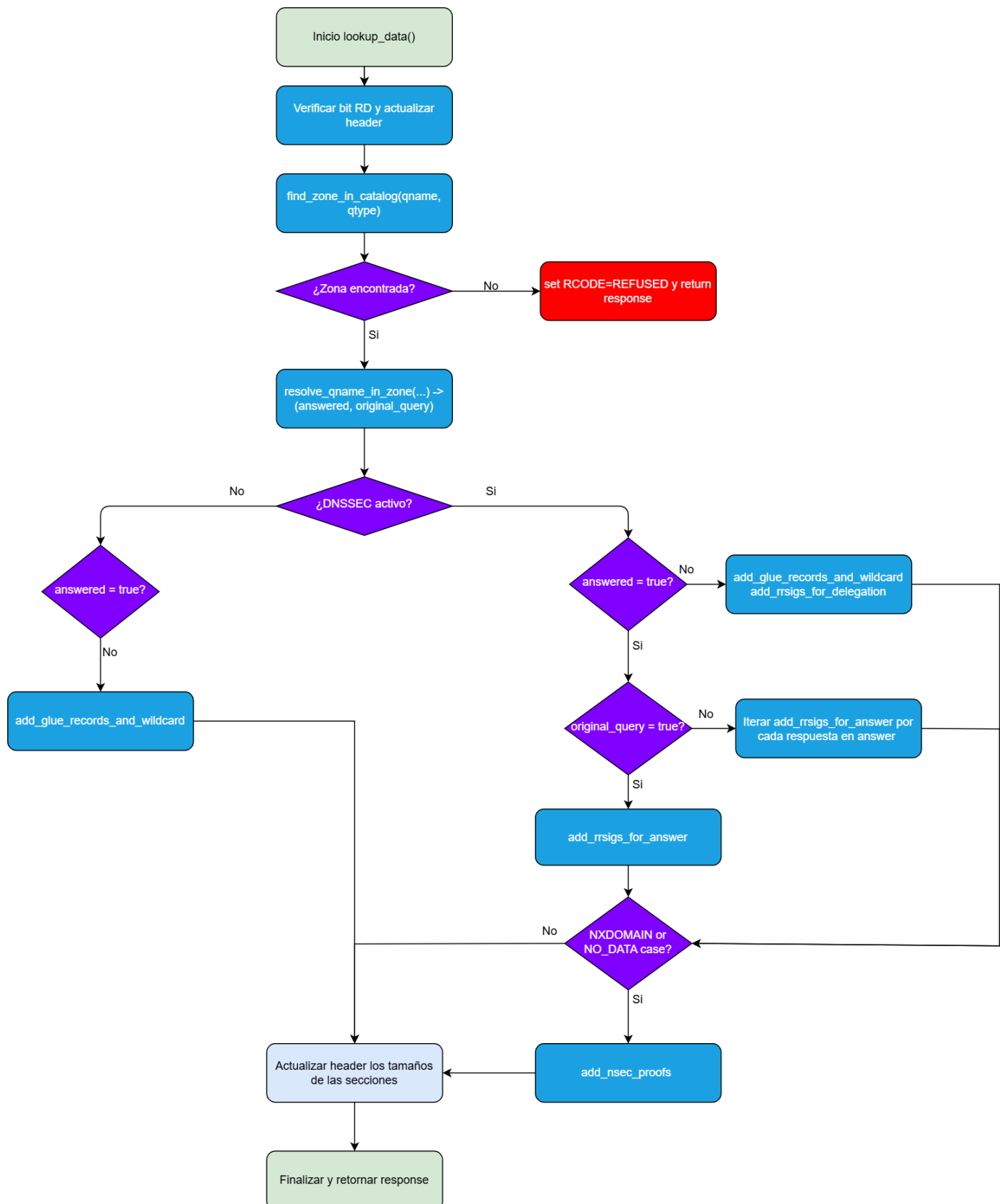
5.1. Flujo y funcionamiento

El algoritmo de búsqueda es el descrito en el RFC 1034 [14] sección 4.3.2, el cual se encarga recibir la consulta DNS y responder acorde a esta, buscando los registros en la base de datos del servidor. Este algoritmo contempla distintos escenarios, desde lo más general hasta lo más específico. Un resumen del mismo es el siguiente:

- Primero, se verifica si el dominio consultado pertenece al catálogo del servidor. Si no pertenece, se responde con el código **REFUSED**, indicando que el servidor no es autoritativo para dicho dominio.
 - Si el dominio consultado y el tipo de registro solicitado existen en la base de datos, se responde directamente con los registros encontrados.
 - Si el tipo de registro solicitado no es **CNAME**, pero existe un registro **CNAME** para el dominio, se realiza una nueva llamada a la función de búsqueda utilizando el valor del **CNAME** como nuevo nombre de dominio y conservando el tipo original. Los registros **CNAME** encontrados se agregan a la respuesta final.
- Si el proceso anterior no produce una respuesta, se continúa con la búsqueda de **glue records** y **wildcards**:
 - Si el dominio posee un **glue record** en la base de datos, se responde con dicho registro.
 - En caso contrario, se verifica si existe un **wildcard** aplicable al dominio consultado.
 - Si se encuentra un **wildcard**, se responde con los registros asociados a él.
 - Si no se encuentra ningún **wildcard**, se responde con **NXDOMAIN**, indicando que el dominio no existe.

Este proceso está simplificado, la versión de código es bastante más compleja y además cubre el estandar de DNSSEC. A continuación se mostrará un diagrama de flujo de la función

lookup_data, la cual es la función principal. Luego en otras subsecciones se explicará en detalle las funciones auxiliares.



La función `lookup_data` recibe como parámetros la consulta DNS completa y la bandera que indica si el modo DNSSEC está activado. A partir de la consulta, se extraen el nombre de dominio y el tipo de registro solicitado, y se inicia el proceso de búsqueda.

Primero, la función intenta localizar la zona correspondiente en el catálogo. Si la zona existe, se procede a resolver el nombre mediante la función `resolve_qname_in_zone`, la cual realiza la resolución directa o a través de registros internos. Esta función retorna dos valores booleanos:

- `answered`: Indica si la consulta fue respondida correctamente.
- `original_query`: Verdadero cuando la respuesta incluye registros del tipo solicitado originalmente, y falso cuando la respuesta se basa en un `CNAME` intermedio.

Si la consulta se realiza con DNSSEC habilitado, se añaden los registros de firma `RRSIG` correspondientes a los registros presentes en la respuesta. En caso de que no se haya podido responder, se agregan los `glue records`, posibles `wildcards` y sus respectivas firmas `RRSIG`. Cuando la respuesta no corresponde a la consulta original (es decir, proviene de `CNAMEs`), se añaden también las firmas `RRSIG` de dichos registros adicionales. Además, si la respuesta corresponde a un error `NXDOMAIN` o a un caso `NO_DATA`, se incluyen los registros `NSEC` que sirven como prueba criptográfica de inexistencia.

Por otro lado, si DNSSEC no está activo, el proceso se simplifica: si no se obtuvo una respuesta directa, se intenta completar la información añadiendo `glue records` o `wildcards`.

Finalmente, se actualizan los contadores y tamaños de las secciones en el encabezado (`HEADER`), y la función retorna la respuesta completa.

A continuación se explican cada una de las funciones auxiliares que componen el algoritmo de búsqueda.

5.1.1. `find_zone_in_catalog`

Esta función recibe el nombre de dominio consultado y el tipo de registro y busca por cada subdominio del dominio original si existe una zona en el catálogo que maneje ese dominio. Por ejemplo si el dominio consultado es `a.niclabs.cl`, la función buscará en el catálogo las zonas `a.niclabs.cl`, `niclabs.cl`, `cl` y `root`. Si encuentra una zona que maneje alguno de estos dominios, retorna la estructura de datos de la zona vista en la sección 4.2.

Esta función es bastante simple pero maneja casos especiales que son importantes y que vienen desde el estándar de DNSSEC. Por ejemplo si tipo consultado es `DS`, la función debe retornar la zona padre, ya que los registros `DS` son manejados siempre por la zona padre. Además existe un problema con el nombre de dominio raíz, ya que como se verá en una sección posterior, el nombre de dominio raíz se maneja internamente como `root` ya que no se puede usar `."` como nombre de carpeta en el sistema de archivos. Por lo tanto, si el dominio consultado es `."` la función debe buscar la zona `root` en el catálogo.

5.1.2. `resolve_qname_in_zone`

Esta función traduce en lógica programática el paso 3.a del algoritmo definido en el RFC 1034 [14], gestionando de forma coherente la búsqueda interna, la detección de alias, las delegaciones y los casos sin datos dentro de una zona DNS.

Se puede dividir el funcionamiento de esta función en los siguientes pasos:

Presencia de un registro CNAME

Si el nombre consultado posee un registro CNAME, la función distingue dos situaciones:

- Si el tipo solicitado **NO** es **CNAME**, el registro **CNAME** se añade igualmente a la respuesta, ya que informa que el nombre consultado es un alias. A continuación, el algoritmo sustituye el nombre original por el destino del **CNAME** y reinicia la búsqueda con ese nuevo nombre.
- Si el tipo solicitado es **CNAME**, se agregan los registros **CNAME** encontrados a la respuesta y se considera la consulta resuelta.

En ambos casos, el bit de respuesta autoritativa (AA) se marca como verdadero, indicando que la información proviene directamente de la zona local.

Coincidencia exacta con nombre y tipo solicitado (distinto a CNAME)

Si el nombre posee registros del tipo consultado, estos se incorporan a la sección de respuestas. En este punto, la función evalúa si los registros corresponden a servidores de nombres (NS). Si el conjunto NS pertenece al apex de la zona (al nombre de la zona, por ejemplo `niclabs.cl`), la respuesta es autoritativa; si pertenece a un subdominio delegado, la función interpreta que se trata de una delegación y desactiva el bit de autoridad. De este modo, el comportamiento refleja la distinción entre datos autoritativos y delegaciones establecida por el RFC.

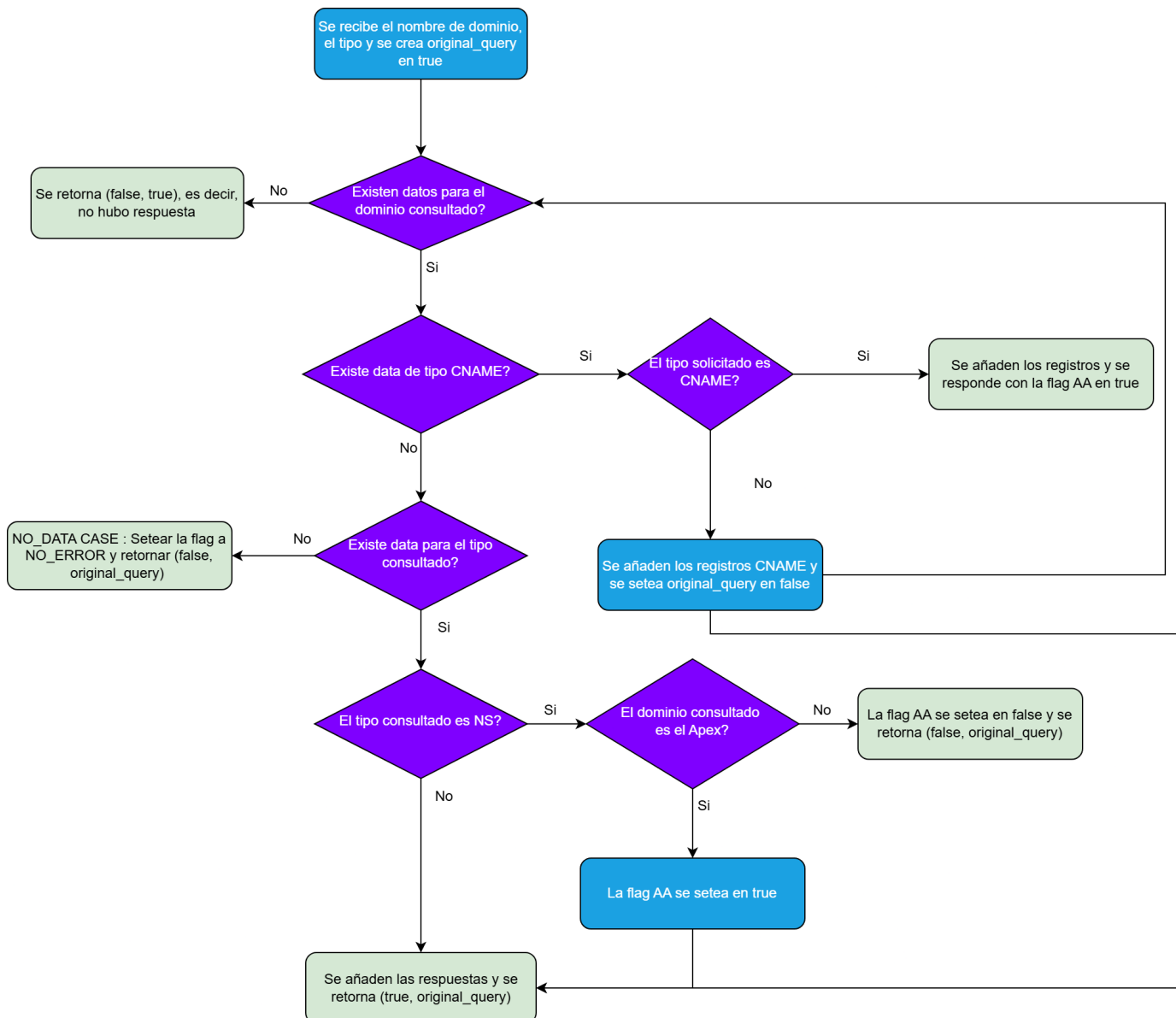
El nombre existe, pero no hay datos del tipo solicitado (NO_DATA)

Cuando el nombre se encuentra en la zona pero no existen registros del tipo pedido, la función retorna con un código de respuesta NOERROR, indicando que el nombre es válido pero no posee información del tipo solicitado. Este caso corresponde al escenario “no data”, en el cual no se genera un error, sino simplemente una respuesta vacía.

El nombre no existe en la zona

Si el nombre de dominio no aparece en los datos de la zona, el algoritmo finaliza sin agregar información a la respuesta. La ausencia total de datos para ese nombre será tratada posteriormente por otra función que añade wildcards y/o NSECs según corresponda. Hasta este punto el código de respuesta debería ser `NXDOMAIN` pero no se asigna aún, ya que podría cambiar si se encuentran `wildcards` o `glue records`.

El diagrama de flujo de esta función es el siguiente:



5.1.3. `add_glue_records_and_wildcard`

La función `add_glue_records_and_wildcard` implementa las secciones 3.b y 3.c del algoritmo de búsqueda. Su objetivo principal es manejar los casos de **delegations** (referrals) y de coincidencias con **wildcards**.

En primer lugar, la función analiza si el dominio consultado (**qname**) corresponde a un punto de delegación. Para ello, busca si existen registros **NS** asociados al nombre, y verifica que dicho conjunto de registros no esté acompañado por un **SOA**. Si existen registros **NS** sin **SOA**, significa que el dominio actual marca el inicio de una subzona delegada, por lo que el servidor debe responder con una **referral**. En este caso:

- Todos los registros **NS** encontrados se copian en la sección **Authority** de la respuesta.
- Luego, por cada nombre de servidor listado en esos registros **NS**, se busca si existe un registro **A** o **AAAA** correspondiente (denominado **glue record**). Si se encuentran, estos se añaden en la sección **Additional**.

Una vez construida la **referral**, el código establece el campo **Rcode** en **NOERROR** y marca la respuesta como no autoritativa (**AA** = 0), dado que el servidor no es responsable de la zona delegada.

Si no se encuentra un punto de delegación, el algoritmo continúa con la búsqueda de coincidencias mediante **wildcards**. Para ello, si existe un registro **RRset** asociado al **wildcard** y que coincida con el tipo de consulta (**qtype**), entonces:

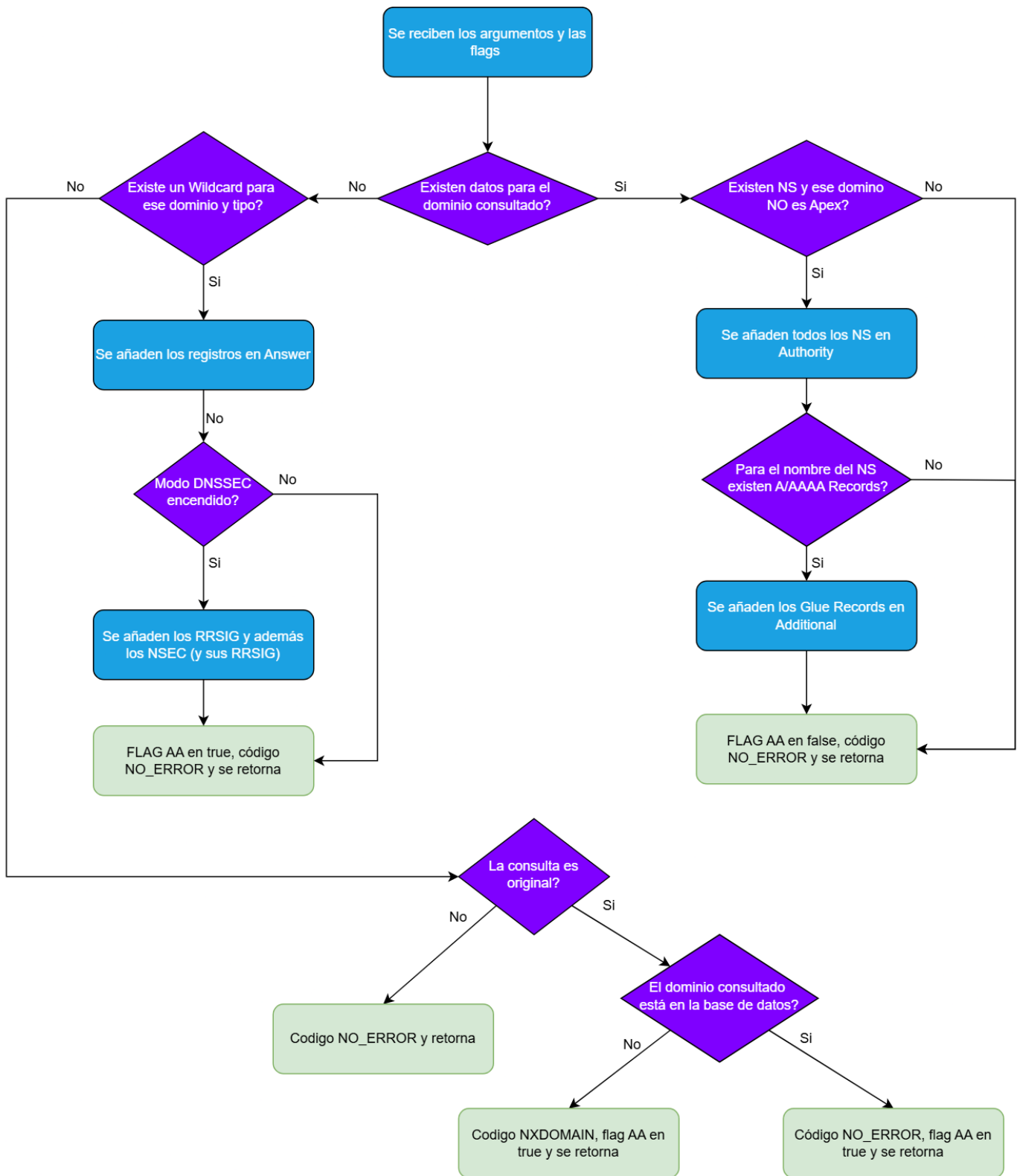
- Se añaden los registros encontrados a la sección **Answer**, utilizando como nombre propietario el **qname** original (según lo especificado por el RFC, aunque la firma se haya generado sobre el **wildcard**).
- Si existen firmas digitales **RRSIG** que cubran el tipo solicitado, estas si corresponde se añaden igualmente en la sección **Answer**.
- Además, se incluyen en la sección **Authority** los registros **NSEC** y sus firmas **RRSIG** asociadas, cuando corresponda.

Finalmente, si no se encuentra ningún **wildcard** aplicable:

- Si la consulta actual corresponde al nombre original, el servidor determina si se trata de un caso **NO_DATA** (nombre existente sin registros del tipo solicitado). En tal caso, responde con **NOERROR** y **AA** = 1.
- En caso contrario, responde con **NXDOMAIN** (nombre inexistente) y también marca la respuesta como autoritativa.

Aclarar que **NO** se añaden los **RRSIG** en los glue record ya que la respuesta **NO** es autoritativa en ese caso.

El diagrama de flujo de la función descrita es el siguiente:



5.1.4. `add_rrsigs_for_delegation`

Esta función se encarga de añadir los registros **DS** y sus correspondientes firmas **RRSIG** a la sección de autoridad del mensaje DNS (**response**) cuando se genera una respuesta de delegación en un servidor DNSSEC.

En términos simples, cuando el servidor responde indicando que un dominio está delegado (por ejemplo, cuando responde con un conjunto de registros **NS** que apuntan al servidor autoritativo de una subzona), esta función busca si en su base de datos también existen los registros **DS** y sus firmas digitales (**RRSIG**) correspondientes, y los incluye en la respuesta.

El envío de los registros **DS** junto con una delegación está descrito en el RFC 4035 [3], sección 2.4. Aclarar también que esta función se ejecuta siempre independientemente de si la hubo o no un referral, ya que si no hay **DS** simplemente se omite su adición. Este aspecto podría ser mejorado en el futuro para evitar llamadas innecesarias a la función.

5.1.5. `add_rrsigs_for_answer`

Esta función se encarga de añadir los registros **RRSIG** correspondientes a un conjunto de registros de respuesta en la sección **Answer** del mensaje DNS (**response**).

En el contexto de DNSSEC, cada conjunto de registros (**RRset**) debe estar acompañado por su firma digital (**RRSIG**), que permite a los resolvers validar criptográficamente que los datos no han sido alterados. La función solo añade los **RRSIG** correspondiente al tipo de consulta original, es decir, si la consulta fue por registros **A**, solo se añadirán los **RRSIG** que firman los registros **A**.

El comportamiento descrito está definido en el RFC 4035 [3], sección 2.2.

5.1.6. `add_nsecs_proofs`

Esta función se encarga de añadir las pruebas criptográficas de inexistencia (**NSEC** y **RRSIG(NSEC)**) a la sección **Authority** del mensaje DNS (**response**) cuando el servidor genera una respuesta de tipo **NO_DATA** o **NXDOMAIN** bajo DNSSEC.

Para dar contexto, los registros **NSEC** son registros especiales en DNSSEC que indican qué nombres de dominio y tipos de registros existen dentro de una zona firmada. Para hacer esto, cada registro **NSEC** enlaza un nombre de dominio con el siguiente en orden canónico, y contiene un mapa de bits que indica qué tipos de registros están presentes para ese dominio. Un orden canónico es un orden lexicográfico de los nombres en minúsculas.

Por ejemplo, si se tiene un registro **NSEC** que indica que el dominio **example.com** tiene registros **A** y **MX**, y enlaza con **foo.example.com**, esto significa que **example.com** existe y tiene esos tipos de registros, y que el siguiente dominio en orden canónico es **foo.example.com**, por lo tanto cualquier nombre entre **example.com** y **foo.example.com** no existe en la base de datos.

La función maneja los siguientes dos casos principales:

Caso 1: el dominio existe, pero el tipo de registro no (respuesta NO_DATA)

Si el dominio consultado está presente en la base de datos, pero no tiene registros del tipo solicitado (`qtype`), entonces se deben incluir los registros `NSEC` y `RRSIG(NSEC)` asociados a ese mismo dominio en la sección `Authority`.

Estos registros demuestran que, aunque el nombre existe, no hay ningún registro del tipo solicitado (por ejemplo, no existe un `MX` para ese dominio). Esto se hace a través del mapa de tipos de registro contenidos en el registro `NSEC`, que indica qué tipos de registros están presentes para ese dominio.

Caso 2: el dominio no existe (respuesta NXDOMAIN)

Si el dominio consultado no está en la base de datos, se necesita saber cual registro `NSEC` cubre el rango que incluye el nombre inexistente.

Para ello, se realiza una búsqueda binaria en la lista de nombres `NSEC`:

- Si el dominio es menor que todos los nombres existentes, se utiliza el último nombre de la lista (por la naturaleza circular del espacio de nombres DNSSEC).
- En caso contrario, se toma el nombre inmediatamente anterior al dominio consultado.

Aclarar que la comparación entre dominios se realiza en su forma canónica.

Luego, se insertan en la respuesta los registros `NSEC` y `RRSIG(NSEC)` del propietario determinado. El uso de los registros `NSEC` para demostrar la inexistencia de nombres o tipos de registros está definido en el RFC 4035 [3], sección 3.1.3

A continuación se mostrarán respuestas del algoritmo para distintos casos de prueba.

5.2. Resultados según cada caso de consulta

A continuación se muestran ejemplos de respuestas generadas por el algoritmo de búsqueda para distintos casos de consulta. Cada ejemplo incluye los registros presentes en la base de datos, la consulta realizada y la respuesta generada en el formato del comando `dig`.

Aclarar que la sección `Additional` no se añadirá en los ejemplos a menos que contenga registros, para simplificar la visualización.

5.2.1. Consulta con coincidencia exacta con nombre de dominio y tipo

Nombre de dominio	Clase	Tipo	Data
www.example.com.	IN	A	0.0.0.0
www.example.com.	IN	A	0.0.0.1

Al preguntar directamente por la IP de **www.example.com** se encuentra una coincidencia exacta con el nombre y el tipo solicitado en los datos, por lo que la respuesta DNS tendrá todos los RRs del RRset en la sección **Answer**, además de los RRSIG correspondientes:

```
1 ;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 4660
2 ;; flags: qr aa; QUERY: 1, ANSWER: 3, AUTHORITY: 0, ADDITIONAL: 1
3
4 ;; QUESTION SECTION:
5 ;www.example.com.          IN      A
6
7 ;; ANSWER SECTION:
8 www.example.com.  3600    IN      A      192.0.2.1
9 www.example.com.  3600    IN      A      192.0.2.2
10 www.example.com.  3600    IN      RRSIG   A 8 3 3600 0 0 12786 www.
    example.com. (keydata ...)
```

5.2.2. Consulta con coincidencia solo en el nombre de dominio, pero existe un registro CNAME

Nombre de dominio	Clase	Tipo	Data
blog.example.com	IN	CNAME	blog2.example.com.
blog2.example.com.	IN	A	1.2.3.4

Al consultar por **blog.example.com** el algoritmo encuentra un registro de tipo **CNAME** y lo guarda en la sección **Answer**. Luego, ejecuta nuevamente el algoritmo utilizando el nombre de dominio especificado en el **CNAME**, logrando así encontrar el registro con la dirección IP correspondiente.

```

1 ;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 4660
2 ;; flags: qr aa; QUERY: 1, ANSWER: 4, AUTHORITY: 0, ADDITIONAL: 1
3
4 ;; QUESTION SECTION:
5 ;blog.example.com.          IN      A
6
7 ;; ANSWER SECTION:
8 blog.example.com.  3600    IN      CNAME    blog2.example.com.
9 blog2.example.com. 3600    IN      A        1.2.3.4
10 blog.example.com.  3600    IN      RRSIG    CNAME 8 3 3600 0 0 12786
      blog.example.com. (keydata ...)
11 blog2.example.com. 3600    IN      RRSIG    A 8 3 3600 0 0 12786
      blog2.example.com. (keydata ...)

```

5.2.3. Consulta que sale de la zona autoritativa : Delegation con Glue Records

Nombre de dominio	Clase	Tipo	Data
b.example.com	IN	NS	ns.a.b.example.com.
ns.a.b.example.com.	IN	A	4.3.2.1

Al consultar por la IP (por el tipo **A**) de **b.example.com**, se encuentra un registro de tipo **NS**. Al comprobar que no existe ningún registro **SOA** en ese dominio (lo que indica que el servidor no es autoritativo en este caso), se debe guardar el registro **NS** en la sección **Authority**. Además, se verifica si existe un registro **glue** correspondiente y, de ser así, se añade en la sección **Additional**.

También se añaden los registros **DS** y sus firmas **RRSIG** correspondientes en la sección **Authority**, esto ya que se está delegando a una subzona. Asimismo se añade el registro **NSEC** y su firma para confirmar la inexistencia del registro **A** solicitado a través del mapa de bits.

```

1 ;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 4660
2 ;; flags: qr; QUERY: 1, ANSWER: 0, AUTHORITY: 5, ADDITIONAL: 2
3
4 ;; QUESTION SECTION:
5 ;b.example.com.          IN      A
6
7 ;; AUTHORITY SECTION:
8 b.example.com.          3600    IN      NS      ns.a.b.example.com.
9 b.example.com.          3600    IN      DS      12345 8 2 (digest_data
10 ... )
11 b.example.com.          3600    IN      RRSIG   DS 8 3 3600 0 0 12786 b.
12 example.com. ( keydata ... )
13 b.example.com.          3600    IN      NSEC    ns1.example.com. NS DS
14 NSEC
15 b.example.com.          3600    IN      RRSIG   NSEC 8 3 3600 0 0 12786
16 example.com. ( keydata ... )
17
18 ;; ADDITIONAL SECTION:
19 ns.a.b.example.com. 3600    IN      A      4.3.2.1

```

5.2.4. Consulta sin coincidencia en el nombre de dominio pero existe un wildcard

Nombre de dominio	Clase	Tipo	Data
*.example.com.	IN	A	0.0.0.0

Al consultar por cualquier dominio que no exista en los datos pero sea subdominio de **example.com**, como por ejemplo **unknown.example.com**, se usará el **wildcard** y se guardará la respuesta en la sección **Answer**. Además se añaden los registros **NSEC** y **RRSIG(NSEC)** correspondientes en la sección **Authority** para probar la inexistencia del nombre consultado.

```

1 ;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 4660
2 ;; flags: qr aa; QUERY: 1, ANSWER: 2, AUTHORITY: 2, ADDITIONAL: 1
3
4 ;; QUESTION SECTION:
5 ;unknown.example.com.      IN      A
6
7 ;; ANSWER SECTION:
8 unknown.example.com.  3600    IN      A      0.0.0.0
9 unknown.example.com.  3600    IN      RRSIG   A 8 2 3600 0 0 12786
   example.com. ( keydata ... )
10
11 ;; AUTHORITY SECTION:
12 *.example.com.      3600    IN      NSEC     b.example.com. A RRSIG
   NSEC
13 *.example.com.      3600    IN      RRSIG   NSEC 8 3 3600 0 0 12786
   example.com. ( keydata ... )

```

5.2.5. Consulta sin coincidencia en nombre de dominio y sin wildcard: NXDOMAIN

Acá no se muestra lo que hay en la base de datos, ya que el cliente consultará por la IP de un dominio que no existe (`unknown.example.com`) y no hay wildcard que aplique. Para lograr esta respuesta se borró temporalmente el wildcard de los datos.

```

1 ;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 4660
2 ;; flags: qr aa; QUERY: 1, ANSWER: 0, AUTHORITY: 2, ADDITIONAL: 1
3
4 ;; QUESTION SECTION:
5 ;unknown.example.com.      IN      A
6
7 ;; AUTHORITY SECTION:
8 example.com.      3600    IN      NSEC     b.example.com. NS SOA
   RRSIG NSEC DNSKEY
9 example.com.      3600    IN      RRSIG   NSEC 8 2 3600 0 0 12786
   example.com. ( keydata ... )

```

5.2.6. Consulta sin coincidencia en el tipo: NO_DATA

Nombre de dominio	Clase	Tipo	Data
<code>www.example.com.</code>	IN	A	1.2.3.4

Al consultar por un dominio que sí existe, pero que no posee registros del tipo solicitado, por ejemplo, un MX para `www.example.com`, la respuesta correspondiente es `NO_DATA`. En

este caso, se incluyen en la sección **Authority** los registros **NSEC** y **RRSIG(NSEC)**, los cuales permiten demostrar criptográficamente la inexistencia del tipo consultado mediante el mapa de tipos contenido en el **NSEC**.

```
1 ;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 4660
2 ;; flags: qr aa; QUERY: 1, ANSWER: 0, AUTHORITY: 2, ADDITIONAL: 1
3
4 ;; QUESTION SECTION:
5 ;www.example.com.          IN      MX
6
7 ;; AUTHORITY SECTION:
8 www.example.com.          3600    IN      NSEC      blog.example.com. A
   RRSIG NSEC
9 www.example.com.          3600    IN      RRSIG     NSEC 8 3 3600 0 0 12786
   example.com. ( keydata ... )
```

El código de respuesta es **NO_ERROR** pero no hay registros en la sección **Answer**, siendo esto equivalente a un caso de **NO_DATA**.

Capítulo 6

EDNS0

Extension DNS 0 (EDNS0) es una extensión del protocolo DNS que permite superar algunas de las limitaciones del formato original del DNS, como el tamaño máximo de los mensajes y la capacidad de transportar información adicional. Fue definido en el RFC 2671 [23] y posteriormente actualizado por el RFC 6891 [20]. EDNS0 es un requisito para muchas de las funcionalidades avanzadas del DNS, incluyendo DNSSEC.

6.1. Características principales de EDNS0

Para habilitar EDNS0, los clientes y servidores DNS incluyen un registro especial en sus mensajes DNS llamado **OPT** (Option), el cual va en la sección **Additional**. Este Resource Record (RR) no usa los campos de manera tradicional, sino que los utiliza para transportar información adicional sobre las capacidades y opciones del DNS. Específicamente, cada campo se utiliza de la siguiente manera:

- **NAME**: Siempre es un nombre vacío (0 bytes).
- **TYPE**: Siempre es 41, que indica que es un registro OPT.
- **CLASS**: Indica el tamaño máximo de mensaje que el emisor/cliente puede manejar. Por ejemplo, un valor de 4096 bytes permite mensajes DNS más grandes que el límite original de 512 bytes.
- **TTL**: Se utiliza para transportar varias opciones y banderas.
 - **Extended RCODE** : Los primeros 8 bits representan el RCODE extendido.
 - **EDNS Version** : 8 bits que representan la versión de EDNS0 (actualmente siempre es 0).
 - **Z field** : 16 bits reservados para banderas, específicamente el bit 15 es la flag DO (DNSSEC OK).

La flag más importante es el DO (DNSSEC OK), que indica si el emisor puede manejar respuestas DNSSEC.

- **RDLLENGTH**: Indica la longitud de los datos de opciones adicionales.
- **RDATA**: Contiene las opciones adicionales que pueden ser utilizadas para diversas funcionalidades, como la negociación de capacidades, pero no se utiliza para DNSSEC.

6.2. Ejemplo de mensaje DNS con EDNS0

Para facilitar el proceso de construcción de una consulta con EDNS0 se creó una función especial llamada `create_dns_opt_query`, la cual recibe como parámetros el nombre de dominio, el tipo de consulta y retorna una consulta DNS que incluye un registro OPT en la sección **Additional**, usando como tamaño máximo de mensaje 4096 bytes.

Un ejemplo del registro OPT generado por esta función es el siguiente:

```
1 [ResourceRecord { name: DomainName { name: '.'}, rtype: OPT,
  rclass: UNKNOWN(4096), ttl: 32768, rdlength: 0, rdata: OPT(
  OptRdata { option: [] }) }]
```

En este ejemplo, se puede observar que el campo `rclass` tiene un valor de `UNKNOWN` con un 4096 dentro, indicando que el emisor puede manejar mensajes DNS de hasta 4096 bytes. Además se puede ver que el campo `ttl` tiene un valor de 32768, lo que indica que la flag DO (DNSSEC OK) está activada

```
1 TTL (32 bits): 00000000 00000000 10000000 00000000
2 | <- RCODE -> | <- VERS -> | <- - - - - Z - - - - -> |
```


Capítulo 7

DNSSEC

En este capítulo se verá cómo funciona DNSSEC dentro del servidor, desde la generación de las claves criptográficas, pasando por distintos procesos de firmas para cada **RRSET**, hasta la creación e inserción de los registros **NSEC** y **DS**.

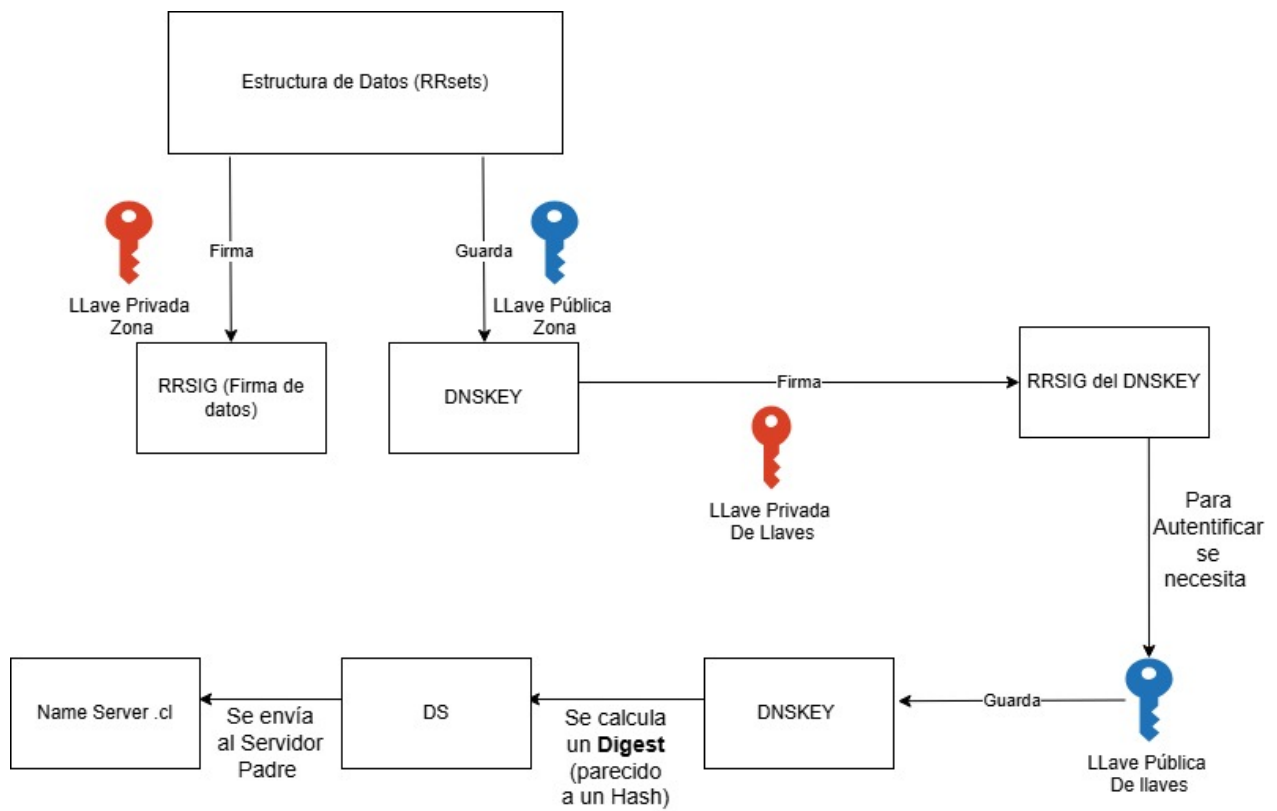
Pero para poder comprobar que las respuestas DNS son auténticas y válidas, es necesario tener un cliente que pueda validar las respuestas DNSSEC, específicamente usando la cadena de autenticación. En primera instancia, esta verificación se iba a realizar dentro del **stub-resolver** de la librería **dns-rust**, pero debido a que las funciones de parsing de la librería están rotas (las funciones **to_bytes** y **from_bytes**), se decidió hacer todo de manera local. En la sección 8.3.1 se explicará más adetalle el porqué las funciones de parsing tienen errores.

7.1. Servidor

7.1.1. Generación de llaves criptográficas

El proceso de firma es complejo y requiere de dos pares de llaves criptográficas por cada zona: un par de llaves para firmar los datos (llaves **ZSK** : Zone Signing Key) y otro par de llaves para firmar las llaves **ZSK** (llaves **KSK** : Key Signing Key). Esto es requerido ya que DNSSEC utiliza un modelo de confianza basado en la jerarquía del DNS, donde cada zona firma sus registros **DNSKEY** con su llave **KSK**, y la zona padre guarda un registro **DS** que apunta a la llave **KSK** de la zona hija, firmando este registro con su propia llave **KSK**. De esta manera, se puede construir una cadena de confianza desde la raíz del DNS hasta cualquier zona, verificando las firmas en cada nivel.

Para mayor claridad se muestra el siguiente diagrama que ilustra el proceso de firmas en DNSSEC:



El proceso de generación de las llaves criptográficas del sistema se realiza mediante el algoritmo RSA, utilizando la librería `rsa` del lenguaje Rust. Como este proceso debe hacerse una sola vez (y no cada vez que se ejecuta el servidor) se separó la lógica dentro de un archivo aparte llamado `generate_keys.rs`, archivo que se encuentra dentro del directorio `src/bin`, para que así pueda ser ejecutado de manera independiente.

El proceso de generación de las llaves criptográficas se realiza de la siguiente manera:

- Se tiene una lista de zonas de los cuales se quieren generar las llaves criptográficas. En este caso, los dominios son `example.com`, `niclabs.cl`, `.com`, `.cl` y `root` (raíz).
- Por cada dominio, se generan los dos pares de llaves criptográficas (KSK y ZSK) utilizando la función `generate_keypair`
- Cada llave generada se guardan archivos dentro del directorio `keys/zona` a través de la función `write_keys_to_disk`

`generate_keypair`

En primer lugar, la función `generate_keypair` recibe como parámetro el tamaño de la llave en bits (por ejemplo, 1024 o 2048). Dentro de la función, se instancia un generador de números aleatorios seguro del sistema operativo (`OsRng`), el cual provee la entropía necesaria

para la generación de claves criptográficamente seguras, a partir de este generador se crea una llave privada RSA.

Posteriormente, la llave pública se deriva directamente desde la llave privada usando el método `to_public_key()`. El resultado de esta función es una tupla que contiene ambas llaves: `(RsaPrivateKey, RsaPublicKey)`.

write_keys_to_disk

La función `write_keys_to_disk` se encarga de almacenar físicamente las llaves generadas en disco. Esta función recibe como parámetros el nombre de la zona (por ejemplo, `c1`) y los pares de llaves correspondientes al ZSK y KSK.

Dentro de la función, primero se crea un directorio con la estructura `keys/domain` para asegurar que exista la carpeta donde se guardarán los archivos PEM (Privacy Enhanced Mail), formato comúnmente utilizado para almacenar llaves criptográficas. Posteriormente, se escriben las llaves en formato PKCS#8 (estándar de representación de llaves en formato PEM) mediante los métodos `to_pkcs8_pem()` para las llaves privadas y `to_public_key_pem()` para las llaves públicas.

Cada llave se almacena en un archivo independiente con nombres descriptivos:

- `zsk_priv.pem`
- `zsk_pub.pem`
- `ksk_priv.pem`
- `ksk_pub.pem`

Aclarar que un nombre de carpeta **NO** puede ser "." (punto), por lo menos en Windows, ya que genera un error al intentar crear el directorio, esto obliga a renombrar la carpeta de la raíz a `root`

7.1.2. Estructuras de datos para las llaves

Una vez generadas las llaves y guardadas en formato PEM dentro de la carpeta `keys/zona` correspondiente, es necesario cargarlas en memoria a través de alguna estructura global accesible desde cualquier parte del servidor. Para esto, se definieron las siguientes estructuras de datos

ZoneKeyStore

La estructura `ZoneKeyStore` es un mapa hash que asocia cada zona (representada como un string) con un puntero a su correspondiente conjunto de llaves criptográficas, que a su vez

es otra estructura llamada `ZoneKeys`. Esta estructura se define de la siguiente manera:

```
1 pub struct ZoneKeyStore {
2     keys: HashMap<String, Arc<ZoneKeys>>,
3 }
```

Se usa `Arc` (Atomic Reference Counting) para permitir que múltiples partes del programa puedan compartir acceso a las llaves de una zona sin necesidad de clonarlas.

La variable global accesible desde cualquier parte del servidor se define como:

```
1 pub static GLOBAL_ZONE_KEY_STORE: Lazy<ZoneKeyStore> = Lazy::new
2 (|| {
3     let mut store = ZoneKeyStore::new();
4
5     let domains = ["example.com.", "niclabs.cl.", "com.", "cl.",
6 "root."];
7
8     for domain in &domains {
9         let path = format!("keys/{}", domain);
10        let zone_keys = ZoneKeys::load_from_path(&path);
11        store.insert_zone_keys(domain.to_string(), Arc::new(
12zone_keys));
13    }
14
15    store
16});
```

La cual basicamente itera sobre la lista de zonas, carga las llaves desde el disco usando la función `ZoneKeys::load_from_path` y las inserta en el `ZoneKeyStore`.

ZoneKeys

La estructura `ZoneKeys` almacena las cuatro llaves criptográficas necesarias para una zona DNSSEC: la llave privada y pública del ZSK y la llave privada y pública del KSK. Esta estructura se define de la siguiente manera:

```
1 pub struct ZoneKeys {
2     zsk_priv: RsaPrivateKey,
3     zsk_pub: RsaPublicKey,
4     ksk_priv: RsaPrivateKey,
5     ksk_pub: RsaPublicKey,
6 }
```

Su metodo `load_from_path` se encarga de leer los archivos PEM desde el disco, parsearlos con el método `from_pkcs8_pem` de la librería `rsa` y crear una instancia de `ZoneKeys` con las llaves correspondientes.

7.1.3. Inserción de DNSKEY's RRSETS en la estructura de datos

Una vez que ya se tienen las llaves cargadas en memoria, se pueden utilizar para crear los registros DNSKEY necesarios para cada zona. Estos registros contienen las llaves públicas utilizadas para verificar las firmas de los registros DNS, tanto del par ZSK como del par KSK.

Por lo que se debe insertar un RRSET de tipo DNSKEY por cada estructura de datos de cada zona. Esto se realiza en la función `insert_dnskey_rrsets`.

El algoritmo `insert_dnskey_rrset` crea un `RRset` para la zona con clase IN, tipo DNSKEY y el TTL dado por el proceso de parsing del masterfile. Luego obtiene las claves desde la variable global `GLOBAL_ZONE_KEY_STORE`, codifica las claves públicas con `encode_rsa_public_key` y construye dos `DnskeyRdata`, los cuales tienen los siguientes campos:

- **flags**: Indica el tipo de llave. El valor 256 indica una llave ZSK y el valor 257 indica una llave KSK. En este caso se usan ambos valores.
- **protocol**: Siempre es 3, según lo especificado en el RFC 4034 [2].
- **algorithm**: Indica el algoritmo criptográfico utilizado. En este caso, se usa el valor 8, que corresponde a RSA/SHA-256.
- **public_key**: Contiene la representación codificada de la llave pública en formato wire.

En cuanto al proceso de codificación de las llaves públicas RSA en el formato requerido por DNSSEC, se utiliza la función `encode_rsa_public_key`, la cual se encarga de codificar una clave pública RSA en el formato wire.

Primero obtiene los bytes del exponente (**e**) y del módulo (**n**) en orden big-endian. Luego, si el exponente tiene una longitud entre 1 y 255 octetos, este se representa con un solo byte; si supera los 255 bytes, se antepone un byte con valor 0 seguido de dos bytes que indican su longitud.

Después de codificar la longitud, se concatenan los bytes del exponente y del módulo, formando la representación final que sigue la sintaxis del campo `public_key` en los registros DNSKEY

Finalmente se insertan los datos a las estructuras correspondientes.

7.1.4. Inserción de los RRSIG de los DNSKEY's RRSET

Una vez insertados los registros DNSKEY en la estructura de datos, se procede a firmarlos utilizando la llave KSK. Esta firma es distinta de las que se aplican a los demás `RRsets` de la zona, los cuales se firman con la llave ZSK. La función `insert_rrsig_rrset_dnskey` es la encargada de realizar este proceso.

En primer lugar, el algoritmo obtiene el conjunto de registros `DNSKEY` desde la estructura de datos y recupera la llave privada `KSK` almacenada en `GLOBAL_ZONE_KEY_STORE`. Luego, calcula el `key_tag` de la `KSK` aplicando el algoritmo definido en el Apéndice B del RFC 4034 [2], el cual consiste en sumar los bytes del `RDATA` del registro `DNSKEY` en formato wire y reducir el resultado a 16 bits. Este valor actúa como un identificador único de la clave utilizada en la firma.

Posteriormente, se construye la estructura `RRSIG_Rdata`, completando los campos definidos en el RFC 4034 [2], sección 3.1. Cada uno de estos campos cumple una función específica dentro del registro de firma:

- **type_covered**: indica el tipo de registro DNS cubierto por la firma. En este caso, su valor es `DNSKEY`, ya que la firma se aplica sobre el conjunto de claves públicas del dominio.
- **algorithm**: especifica el algoritmo criptográfico utilizado para generar la firma. Se asigna el valor 8, correspondiente a `RSASHA256`.
- **labels**: representa la cantidad de etiquetas (labels) del nombre de dominio al que pertenece el `RRSet`. Su valor se obtiene mediante la función auxiliar `count_labels(zone_name)`.
- **original_ttl**: contiene el valor TTL original del `RRSet` firmado, copiado directamente desde el conjunto `DNSKEY`.
- **signature_inception** y **signature_expiration**: definen el intervalo de validez de la firma. En esta implementación se establecen con valor 0, ya que el servidor no implementa aún manejo de tiempo real ni soporte para zonas dinámicas.
- **key_tag**: identifica de forma única la clave usada para la firma. En este caso se utiliza el valor calculado previamente a partir de la `KSK`.
- **signer_name**: indica el nombre del propietario de la clave utilizada para la firma, correspondiente al dominio de la zona (`zone_name`).
- **signature**: almacena la firma criptográfica generada sobre el `RRSet` de `DNSKEY`. A continuación se detalla el proceso para calcular este campo.

Para generar la **signature** se necesita firmar el TBS (To Be Signed), el cual sería la concatenación de los campos del `RRSIG` (exceptuando la misma **signature**) con el `DNSKEY` en versión canónica. Este proceso se realiza mediante la función auxiliar `build_tbs_data` siguiendo el orden establecido por el proceso de canonicalización de DNSSEC.

Finalmente, se utiliza la llave privada del par `KSK` para firmar el TBS utilizando el algoritmo RSA con SHA-256 y se añade el `RRSET` de `RRSIG` resultante a la estructura de datos del servidor.

7.1.5. Creación e inserción de firmas y el manejo de casos bordes

Después de insertar y firmar los registros `DNSKEY`, el siguiente paso consiste en firmar todos los demás `RRSET` presentes en la zona. Naturalmente, los registros `RRSIG` no deben firmarse, ya que ellos mismos contienen las firmas de los conjuntos de registros.

Este procedimiento es prácticamente idéntico al proceso de firma de los `DNSKEY`, con la diferencia de que en este caso se utiliza la llave `ZSK` en lugar de la `KSK`, y se aplica a cada `RRSET` perteneciente a un dominio de la zona. Sin embargo, existen ciertos casos borde que deben manejarse cuidadosamente: los registros `NSEC`, los registros no autoritativos o de delegación, y los `wildcards`.

En primer lugar, el proceso de generación y firma de los registros `NSEC` es más extenso y se detalla en la siguiente sección. En cuanto a los registros no autoritativos o de delegación (por ejemplo, los registros `NS` que apuntan a una zona hija), estos no deben ser firmados, ya que son responsabilidad de la zona delegada. Para detectar estos casos dentro de la estructura de datos, se implementó la función `is_authoritative_rrset`, cuyo flujo general es el siguiente:

- Verifica que el nombre del `RRset` pertenezca efectivamente a la zona. Si no está dentro de la zona, no se considera autoritativo.
- Evita firmar los `RRset` de delegación, es decir, los registros `NS` que no se encuentran en el apex de la zona.
- Evita firmar los `glue records`, es decir, registros `A` o `AAAA` asociados a una delegación, los cuales solo sirven para resolver los nombres de los servidores delegados.
- En cualquier otro caso, el `RRset` se considera autoritativo y debe ser firmado.

En el caso de los `wildcards`, su identificación es directa, ya que el nombre del `RRset` comienza con un asterisco (*). Cuando esto ocurre, se debe restar uno al conteo de etiquetas (`labels`) del campo correspondiente en el `RRSIG`. Esto permite que el resolver pueda reconocer que la respuesta proviene de un `wildcard` y validar correctamente su firma. De no hacerlo, el cliente no podría distinguir el `wildcard` de un nombre explícito y la validación `DNSSEC` fallaría.

Esta regla está establecida explícitamente en el RFC 4035 [3], Sección 5.3.4

7.1.6. Creación e inserción de los registros `NSEC`

Los registros `NSEC` (Next Secure) son fundamentales en `DNSSEC` para proporcionar autenticación de ausencia de datos. Estos contienen los siguientes campos:

- `next_name`: Indica el siguiente nombre de dominio en orden canónico dentro de la zona.
- `type_bitmap`: Es un mapa de bits que señala los tipos de registros presentes para el nombre de dominio actual.

La función `create_and_insert_nsec_rrsets` construye e inserta todos los `NSEC RRSET` de la zona. El proceso completo involucra dos tareas principales:

Construcción de la lista de nombres para NSEC (NSEC owner names)

El primer paso consiste en determinar exactamente cuáles nombres del archivo de zona deben poseer un `NSEC`. Esto se realiza mediante la función `gather_nsec_names`, la cual recorre toda la estructura interna de datos y selecciona únicamente aquellos nombres que corresponden a información autoritativa o a puntos de delegación. Esta filtración es esencial porque los registros `NSEC` describen únicamente nodos válidos dentro de la jerarquía DNS de la zona firmada. Una vez reunidos estos nombres, la lista se ordena en el orden canónico, siguiendo lo estipulado en el RFC 4035 [3], sección 2.3.

Construcción de los mapas de tipos

Posteriormente, para cada nombre de esta lista se construye el campo `type_bit_maps`. Durante la iteración de los tipos presentes en un nombre, la implementación evita incluir todos los tipos considerados “pseudo-tipos”, tales como `OPT`, `TSIG`, `ANY`, entre otros. Estos tipos no forman parte de la información firmada ni pueden aparecer legítimamente en un bitmap de un `NSEC`, tal como lo establece el RFC 4034 [2] 4.1.2.

Luego, dependiendo del tipo de nodo, existen reglas especiales. Si el nombre es un punto de delegación (es decir, contiene un `RRset NS` pero no es el apex), solo se deben incluir en el bitmap los tipos `NS` y `DS`. Esto se debe a que el servidor padre no es autoritativo para el resto de los registros del dominio delegado y, por lo tanto, no debe afirmar su existencia mediante un `NSEC`. En contraste, si el nombre corresponde a información completamente autoritativa, se incluyen todos los tipos válidos presentes en el nodo. En ambos casos, de manera obligatoria, se agrega también el tipo `NSEC` al bitmap, ya que cada nombre que tiene un `NSEC` necesariamente debe anunciarlo explícitamente.

Finalmente, con el nombre actual, el siguiente nombre en orden canónico y el bitmap resultante, se construye la `RData NSEC`. El TTL del registro se toma desde el campo “minimum TTL” del `SOA` del apex. Con toda esta información se produce un nuevo `RRset` de tipo `NSEC`, el cual se inserta en la estructura de datos de la zona.

7.1.7. Inserción de RRSIGS para los NSEC RRSET

El proceso de construcción de los registros `NSEC` es, en esencia, el mismo que el descrito previamente en la sección 7.1.5. Sin embargo, surge una pregunta natural: ¿Por qué las firmas `RRSIG` correspondientes a los `NSEC` se generan en un paso separado, en lugar de ser firmadas junto con el resto de los `RRSET`? La razón es estructural, ya que los registros `NSEC` deben reflejar con precisión todos los tipos existentes en cada nombre, y eso incluye los `RRSIG`. En otras palabras, el mapa de tipos (`type_bit_maps`) de un `NSEC` debe considerar la presencia de las firmas ya generadas, por este motivo, los `RRSIG` de los demás `RRSET` deben existir

antes de crear los **NSEC**. Si se intentara firmar todo en un solo paso, los **NSEC** no podrían incluir los tipos correctos, produciendo un conjunto inconsistente e inválido bajo **DNSSEC**. Debido a esta dependencia lógica, la firma de los **NSEC** se ejecuta necesariamente como una fase posterior y separada.

7.1.8. Creación del **RRSET DS** para la zona padre

Un registro **DS** (Delegation Signer) es un tipo especial de registro **DNS** que se utiliza en **DNSSEC** para establecer una cadena de confianza entre una zona padre y una zona hija. El cual contiene los siguientes campos:

- **key_tag**: Identificador único de la clave pública de la zona hija.
- **algorithm**: Algoritmo criptográfico utilizado por la clave pública.
- **digest_type**: Tipo de función hash utilizada para generar el digest.
- **digest**: Resultado de aplicar la función hash a la concatenación del nombre canonico de la zona hija y su llave pública.

La construcción del **RRSET DS** se realiza mediante la función `build_ds_rrset`. Para ello primero obtiene el **DNSKEY RRSET** del apex de la zona y prepara un nuevo **RRSET DS** con el mismo nombre, clase y **TTL**. Luego, recorre cada registro **DNSKEY** y selecciona únicamente aquellos que corresponden a la clave **KSK**. Para cada llave calcula el **key tag** y genera el valor del **digest** siguiendo las especificaciones del RFC 4034 [2], sección 5.1.4, concatenando el nombre de dominio en forma canónica con el **Rdata** del **DNSKEY** y aplicando la función hash dada según el **digest_type**. Con esta información se construye la estructura **DsRdata**, que posteriormente se inserta como registro dentro del **DS RRSET**. Finalmente, el **RRSET** resultante se retorna para que pueda ser insertado en la estructura de datos de la zona padre.

El servidor no tiene acceso directo a la zona padre, por lo que al generar el **RRSET DS**, este se imprime en consola, se copian los datos y se insertan manualmente en el master-file de la zona superior. Esto coincide con la práctica actual de los servidores **DNSSEC**, ya que no existe todavía un mecanismo estandarizado y ampliamente adoptado para que los registros **DS** se envíen automáticamente al progenitor.

En la reunión ICANN 84 (octubre de 2025) se abordó precisamente este tema en el contexto del taller “Towards an Industry Best Practice for DS Automation” [21], donde se revisaron estadísticas de adopción y los desafíos operativos para la automatización del envío de **DS**.

7.2. Resolver

Es fundamental verificar que las firmas generadas por el servidor sean correctas dentro del contexto de **DNSSEC**. Para ello, se implementó completamente la lógica de validación de

firmas y de la cadena de autenticación. Originalmente, este proceso iba a realizarse dentro del `stub-resolver` de la librería `dns-rust`; sin embargo, debido a problemas en sus funciones internas de parsing, se optó por efectuar toda la validación localmente, replicando con precisión el flujo que tendría un `stub-resolver` real.

El proceso de autenticación debe distinguir entre varios escenarios posibles en una respuesta DNSSEC:

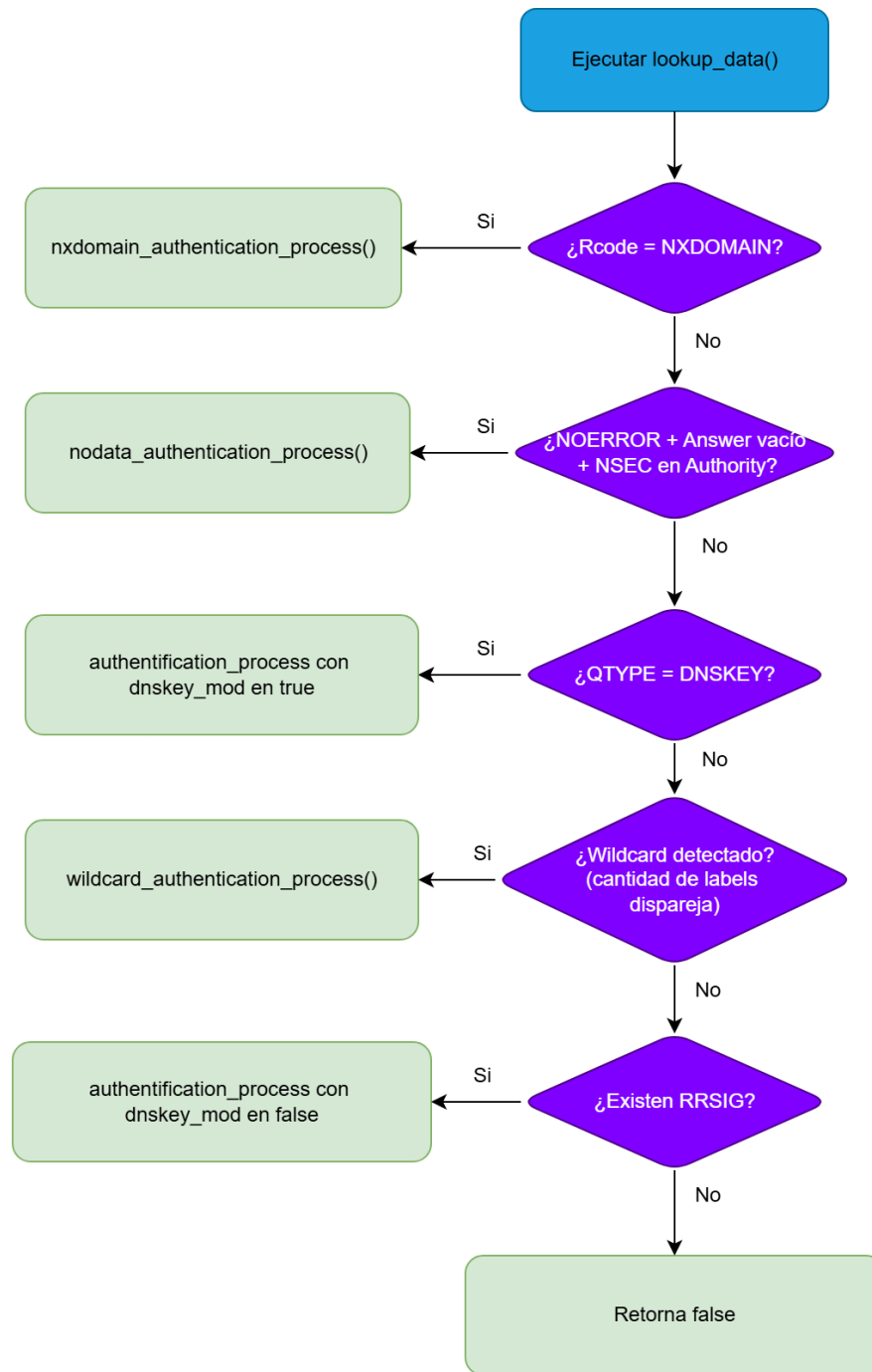
- Respuestas con datos en la sección `Answer` y sus `RRSIG` asociados.
- Respuestas con código `NXDOMAIN`.
- Respuestas `NO_DATA`.
- Respuestas originadas por consultas `DNSKEY`.
- Respuestas generadas a partir de un `Wildcard`.

Para gestionar correctamente cada caso, se definió la función `dnssec_authentication`, la cual recibe la consulta DNS original y retorna verdadero si la respuesta es válida bajo DNSSEC, y falso en caso contrario. El flujo general de esta función es el siguiente:

- En primer lugar, se ejecuta la función `lookup_data` con `DNSSEC` habilitado, obteniéndose así la respuesta completa que será inspeccionada para determinar qué mecanismo de validación corresponde aplicar.
- Si el código de respuesta en el `Header` es `NXDOMAIN`, significa que el nombre consultado no existe en la zona. En este caso se activa el proceso de autenticación basado en pruebas `NSEC` de inexistencia de dominio, mediante `nxdomain_authentication_process`.
- Si el código es `NOERROR`, pero la sección `Answer` está vacía y la sección `Authority` contiene un registro `NSEC`, la respuesta corresponde al caso `NO_DATA`. Para validar esta situación, se ejecuta `nodata_authentication_process`, el cual verifica la ausencia del tipo consultado mediante la prueba `NSEC` apropiada.
- Si la consulta original corresponde al tipo `DNSKEY`, se activa el modo especial de autenticación que utiliza la `KSK`. En este caso se invoca `authentication_process` con la bandera `dnskey_mod` en verdadero.
- Si no se cumplen las condiciones anteriores, se analiza la presencia de registros `RRSIG` en las secciones `Answer` y `Authority`. Luego, se compara la cantidad de labels (etiquetas) del nombre de dominio del `RRset` con el campo `labels` contenido en la firma `RRSIG`. Si el número de labels del dominio es mayor que el valor en el `RRSIG`, se trata de una respuesta producida por `wildcard expansion`. En este caso, se ejecuta `wildcard_authentication_process`.
- Si no se detecta `wildcard`, pero al menos un `RRSIG` existe, la respuesta corresponde a un caso de validación estándar, por lo que se invoca `authentication_process`.

- Finalmente, si la respuesta no contiene ningún RRSIG, no existe información criptográfica que validar. El proceso falla explícitamente retornando falso, pues la respuesta no cumple los requisitos de seguridad DNSSEC.

El siguiente diagrama de flujo ilustra visualmente este proceso:



A continuación se describen todos los procesos de validación para cada caso:

7.2.1. Proceso de autenticación para nombres inexistentes (NXDOMAIN)

La función `nxdomain_authentication_process` se encarga de validar respuestas con código `NXDOMAIN`, asegurando que el nombre consultado no existe en la zona, utilizando los registros `NSEC` presentes en la sección `Authority` de la respuesta DNS.

La función tiene dos verificaciones principales:

Verificación del intervalo NSEC

Itera sobre cada registro `NSEC` en la sección `Authority` y, para cada uno, verifica si el nombre consultado cae dentro del intervalo definido por el nombre dueño del `NSEC` y su `next_name`. Esta verificación se realiza comparando los nombres en su forma canónica. La función utilizada para esto, `name_in_nsec_range`, comprueba si el nombre consultado está estrictamente entre el `owner` y el `next`, o si se trata del caso especial de “wrap-around”, donde el `next` es lexicográficamente menor que el `owner` y el intervalo cruza el final del espacio de nombres. Si el nombre consultado está cubierto por alguno de los intervalos `NSEC`, se continúa retornando verdadero; si ninguno lo cubre, la autenticación retorna falso.

Para ilustrar el caso normal, considere:

`example.com. NSEC foo.example.com.`

Este `NSEC` cubre cualquier nombre estrictamente entre `example.com.` y `foo.example.com.`. Por ejemplo, `bar.example.com.` está dentro del intervalo, retornando verdadero, en cambio `baz.example.com.` no lo está, retornando falso.

En contraste, el caso de `wrap-around` ocurre cuando `next_name` es lexicográficamente menor que `owner`. Por ejemplo:

`z.example.com. NSEC example.com.`

Aquí el intervalo envuelve el final del espacio de nombres, cubriendo todos los nombres mayores que `z.example.com.` (como `zzz.example.com.`) y también todos los menores que `example.com.` (como `a.example.com.`). Si el nombre consultado cae dentro de alguno de estos intervalos `NSEC`, se retorna verdadero, si ninguno lo cubre (como en el caso de `m.example.com.`), se retorna falso.

Verificación de firmas RRSIG del NSEC

Luego se verifican las firmas `RRSIG` asociadas a los `NSEC` encontrados, estas firmas están en la sección `Authority` y el proceso normal es que estén en la sección `Answer`, por lo

que se limpia la sección **Answer** y se copian las firmas **RRSIG** a esta sección para reutilizar la función de autenticación estándar la cual es **authentication_process**. De esta manera, si la firma es válida, se confirma que el **NSEC** es auténtico y por ende el nombre consultado no existe en la zona.

Si ambas verificaciones son exitosas, la función retorna verdadero, confirmando que la respuesta **NXDOMAIN** es válida bajo **DNSSEC**.

7.2.2. Proceso de autenticación para respuestas **NO_DATA**

La función **nodata_authentication_process** valida respuestas DNS que indican la ausencia de datos para un tipo específico asociado a un nombre existente, es decir, respuestas **NO_DATA**. Este caso es similar al proceso de autenticación para **NXDOMAIN**, con la diferencia de que aquí el nombre consultado *sí existe* en la zona, pero el tipo solicitado **no**.

El proceso de validación consta de 4 pasos principales:

Verificación de existencia del nombre

Primero se debe encontrar un registro **NSEC** cuyo **owner name** coincida exactamente con el nombre consultado. Si no se encuentra tal **NSEC**, la autenticación falla inmediatamente, retornando falso, ya que esto indicaría que el nombre no existe en la zona.

Verificación de la ausencia del tipo solicitado

Una vez localizado el **NSEC** correspondiente al nombre consultado, se examina su campo **type_bitmap** para verificar que el tipo solicitado no esté presente. Si el tipo está ausente en el bitmap, se sigue con el proceso, de lo contrario se retorna falso.

Verificación del **DS** asociado si es que existe

Un caso particular que siempre se manifiesta como **NO_DATA** es el de los **delegation points**. En una delegación, la zona padre es autoritativa únicamente para los registros **NS** (y, si existe, **DS**), mientras que **no** es autoritativa para ningún otro tipo perteneciente a la zona hija. Por lo tanto, cualquier consulta hacia el padre que solicite un tipo distinto de **NS** o **DS** debe producir necesariamente una respuesta **NO_DATA**. El nombre existe, pero la zona padre no puede servir el tipo solicitado porque no le pertenece.

En estos casos, el RFC 4035 [3], Sección 2.4, establece que la zona padre *debería* (“**SHOULD**”) incluir un registro **DS** si este está disponible. El propósito de este **SHOULD** es permitir a los validadores confirmar la continuidad de la cadena de confianza hacia la zona delegada. Es decir, si se entrega un **NSEC** para probar la ausencia de un tipo determinado y, adicionalmente,

un DS (con su correspondiente RRSIG) para demostrar que la delegación está correctamente firmada, el validador puede:

- Confirmar que no existen datos del tipo consultado en la zona padre.
- Verificar que la delegación hacia la zona hija mantiene una cadena de confianza válida.

Esta lógica de añadir el DS en estos casos de `delegation points` está cubierta por el algoritmo de búsqueda, específicamente en la función auxiliar `add_rrsigs_for_delegation`, que tiene un mal nombre ya que no solo añade los RRSIG sino que también añade el DS si es que existe. Esto se vió en la sección 5.1.4

Para cumplir con esta recomendación, la función inspecciona la sección **Authority** en busca de registros DS asociados al nombre consultado y, en caso de encontrarlos, los conserva para su validación posterior. Asimismo, verifica que los registros RRSIG que cubren a dichos DS también estén presentes en la respuesta, asegurando así que cualquier información entregada sobre la delegación pueda ser autenticada criptográficamente.

Verificación de firmas RRSIG del NSEC y DS

Finalmente, se verifican las firmas RRSIG asociadas tanto al registro NSEC como, cuando corresponde, al registro DS. El procedimiento es análogo al utilizado en la autenticación de respuestas NXDOMAIN: se limpia la sección **Answer** y se trasladan a ella los registros NSEC, DS y sus respectivas firmas RRSIG. Esto permite reutilizar directamente la función estándar de validación `authentication_process`, garantizando así que todas las pruebas criptográficas asociadas al caso NO_DATA se verifiquen de manera uniforme y consistente.

7.2.3. Autenticación para respuestas provenientes de Wildcards

La función `wildcard_authentication_process` se encarga de validar respuestas DNS que han sido generadas mediante la expansión de un `wildcard` (comodín). Este tipo de respuestas ocurre cuando un nombre no existente coincide con un patrón del tipo `*.example.com`, y el servidor autoritativo genera los registros utilizando dicho `wildcard`.

El proceso de validación consta de los siguientes pasos:

Descubrir cuál fue el wildcard utilizado

El primer paso consiste en identificar el nombre del wildcard original que fue utilizado para generar la respuesta. Para ello, la función inspecciona la sección **Authority** en busca del primer registro NSEC, ya que según el RFC 4035 [3], sección 3.1.3.4, es obligatorio que toda respuesta basada en `wildcard` incluya un registro NSEC que pruebe la inexistencia explícita del nombre consultado.

En una respuesta genuina de `wildcard`, el dueño de ese `NSEC` es precisamente el nombre del `wildcard` (por ejemplo, `*.example.com.`). Este valor es crucial, ya que es el **nombre real** sobre el que se generó y firmó el `RRSIG` del `RRset` en el servidor autoritativo.

Reconstruir la sección `Answer` reemplazando el nombre consultado por el `wildcard`

La sección `Answer` que llega al resolver contiene como `owner name` el nombre consultado por el cliente, por ejemplo `unknown.example.com.`, incluso si dicho nombre no existe. Esto es un comportamiento obligatorio de acuerdo con RFC 1034 [14] 4.3.3.

Sin embargo el `RRSIG` del `RRset` no fue generado sobre `unknown.example.com.` sino sobre el `wildcard *.example.com`. Por lo tanto, para poder verificar correctamente la firma, es necesario reconstruir localmente el `RRset` **tal como fue firmado originalmente**. Esto implica tomar cada registro de la sección `Answer` y reemplazar su nombre dueño por el `wildcard` detectado.

El resultado es una nueva sección `Answer` que refleja el estado real del `RRset` al momento de la firma en el servidor.

Autenticar las firmas del `RRset` reconstruido

Una vez reconstruida la sección `Answer`, se procede a verificar el `RRSIG` asociado mediante la función estándar `authentication_process`. Si esta verificación falla, la respuesta completa debe considerarse inválida, ya que implicaría que el `RRset` entregado no corresponde al `wildcard` que lo generó, o que la firma ha sido alterada.

Validar los `NSEC` asociados

Finalmente, la función verifica que la expansión del `wildcard` se haya producido legalmente y que el nombre consultado realmente **no exista en la zona**. Para esto:

- Se crea una copia del mensaje de respuesta.
- Se limpian las secciones `Answer` y `Additional`, dejando únicamente la sección `Authority`. Dicha sección contiene los `NSEC` y sus correspondientes `RRSIG`, que son los elementos necesarios para demostrar la no existencia del nombre consultado.
- Se ajusta el código de respuesta a `NXDOMAIN`, ya que ahora se está validando la inexistencia del nombre original.

La función `nxdomain_authentication_process` valida entonces esta copia modificada del mensaje.

Si todas las verificaciones son exitosas, la función retorna verdadero, confirmando que la respuesta generada por el `wildcard` es válida bajo DNSSEC.

7.2.4. Proceso de Autenticación DNSSEC Estándar

La función `authentication_process` implementa la validación estándar de firmas DNSSEC para respuestas que contienen datos en la sección `Answer`. De hecho, todas las funciones anteriores de validación (`NXDOMAIN`, `NO_DATA` y `Wildcards`) terminan invocando esta función para validar las firmas `RRSIG` correspondientes. La función toma como argumentos la respuesta DNS completa y una bandera booleana `dnskey_mod` que indica si la consulta original fue de tipo `DNSKEY`. El proceso de validación consta de las siguientes etapas:

Preparación de información y variables iniciales

El algoritmo comienza extrayendo desde la respuesta DNS las secciones `Answer`, `Authority` y `Question`, clasificando los registros en dos grupos:

- **Registros de datos:** Todos los registros que no son `RRSIG`, los cuales contienen la información real solicitada.
- **Registros `RRSIG`:** Registros `RRSIG`, que contienen las firmas digitales.

A partir del nombre de dominio consultado se determinan las dos últimas etiquetas, que constituyen el apex de la zona (por ejemplo, `example.com.` o `niclabs.cl.`) Este apex es fundamental para obtener las claves `DNSKEY` necesarias en las siguientes etapas.

Obtención de las claves `DNSKEY` y sus firmas (`DNSKEY RRset`)

Para poder validar una respuesta DNSSEC, el resolver debe obtener el conjunto de claves `DNSKEY` del apex. Esto lo realiza la función auxiliar `get_pubkeys_dnskey_and_rrsig`, que:

- Crea una consulta DNS de tipo `DNSKEY` dirigida al apex de la zona y la resuelve utilizando la función `lookup_data` con DNSSEC habilitado.
- Identifica y separa las claves públicas según su rol criptográfico: la `ZSK` y la `KSK`.
- Convierte ambas claves a estructuras `RSA` utilizables por el validador.

La función retorna: el par (`ZSK`, `KSK`), el `RRset DNSKEY` y el `RRset RRSIG(DNSKEY)`

Verificación de las firmas de la respuesta

La verificación de firmas sobre los datos recibidos (`Answer + Authority`) se realiza mediante la función `verify_rrsig`. Para cada `RRSIG` se ejecutan los siguientes pasos:

- Determinar el tipo de **RRset** que la firma cubre (**type_covered**).
- Reunir el **RRset** correspondiente desde los registros recibidos.
- Construir el **RRset** en formato canónico DNSSEC.
- Construir la estructura TBS (To-Be-Signed), que combina la metada del **RRSIG** y el **RRset** canónico.
- Verificar la firma usando Hash: SHA-256, Algoritmo: RSA PKCS1 v1.5 y clave pública: ZSK (modo normal) o KSK (si **dnskey_mod** está activo)

Notar que en estricto rigor se debería usar el Hash y Algoritmo indicados en el campo **algorithm** del **RRSIG**, pero en esta implementación solo se soporta RSA-SHA256, ya que es el algoritmo más comúnmente utilizado en la práctica y fue el único implementado en el servidor.

Verificación del **RRset DNSKEY**

Cuando **dnskey_mod** está deshabilitado (modo normal), el algoritmo debe además validar el propio conjunto **DNSKEY**. Esta verificación es crítica, porque el **RRset DNSKEY** es la base criptográfica que permite validar al resto de los **RRsets** de la zona. El proceso consiste en:

- Tomar el **RRset DNSKEY**, su **RRSIG(DNSKEY)** y construir la estructura TBS correspondiente.
- Verificar la firma usando exclusivamente la KSK, ya que solo la KSK puede firmar el **RRset DNSKEY**.

Solo si esta firma es válida, las claves **DNSKEY** pueden considerarse auténticas y confiables para validar el resto de la zona.

Cadena de Autenticación DNSSEC

Para validar completamente la cadena de confianza DNSSEC, el algoritmo necesita:

- Zona hija: el apex (**example.com** o **niclabs.cl**)
- Zona padre: la última etiqueta del apex (**com**, **cl**)
- El **DNSKEY Rdata** de la KSK de la zona hija, ya que el **DS** en la zona padre debe corresponder con ese valor.

Con los datos anteriores, delega la validación de la cadena de autenticación a la función especial **verify_ds_chain**. La cual se verá en el siguiente sección ya que es un proceso complejo que involucra múltiples pasos y consultas.

Resultado final

La respuesta DNS se considera válida únicamente si todas las validaciones anteriores fueron exitosas: las firmas de los `RRsets`, la firma del `RRset DNSKEY` y la cadena de autenticación. Si alguna de estas verificaciones falla, la función retorna falso, indicando que la respuesta no cumple con los requisitos de seguridad DNSSEC.

7.2.5. Proceso de verificación de la cadena de autenticación

La validación de la cadena de autenticación DNSSEC es el mecanismo que permite verificar que la clave pública (KSK) utilizada por una zona hija es auténtica y se encuentra correctamente delegada por su zona padre y así sucesivamente hasta llegar a una autoridad de confianza (la zona `root`). Este proceso es fundamental para garantizar la integridad y autenticidad de las respuestas DNS, ya que asegura que los datos no han sido alterados y provienen de una fuente confiable, obligando que la zona padre respalde la autenticidad de la zona hija.

La función es explicada en las siguientes etapas:

Obtención del registro DS en la zona padre

El procedimiento comienza realizando una consulta DNS de tipo `DS`, utilizando como nombre objetivo el apex de la zona hija. Esta consulta debe ser resuelta por la zona padre, ya que los registros `DS` siempre se almacenan en la delegación que el padre mantiene respecto del hijo.

Es importante señalar que la función `lookup` utilizada para enviar esta consulta incorpora un manejo especial para este caso, tal como se explicó en la sección 5.1.1, si `lookup` tratara esta consulta como una consulta DNS normal, intentaría resolverla dentro de la propia zona hija (pues el nombre coincide con su apex), lo que impediría encontrar el registro `DS` en la zona correcta. Para evitar este error, `lookup` detecta consultas de tipo `DS` y fuerza la resolución hacia la zona padre. Aclarar que este comportamiento especial es debido a que no se habla con otros servidores DNS externos, sino que todo se simula localmente.

Como resultado de esta etapa, se obtiene desde la zona padre: El `RRset DS` y su correspondiente `RRset RRSIG(DS)`.

Obtención del `DNSKEY` `RRset` y sus firmas en la zona padre

Una vez extraído el `DS`, el verificador debe obtener los registros `DNSKEY` de la zona padre junto con sus firmas `RRSIG(DNSKEY)` y el par de llaves públicas (`ZSK`, `KSK`) de dicha zona. Esto se realiza mediante la función auxiliar `get_pubkeys_dnskey_and_rrsig`.

Esta información es necesaria para verificar tanto las firmas del `DS` como la autenticidad

del propio DNSKEY del padre.

Verificación criptográfica en la zona padre

Con toda la información disponible, se realizan dos verificaciones fundamentales:

- **Verificación del RRset DNSKEY del padre con su propia KSK:** El RRset DNSKEY debe estar firmado con la KSK de la zona padre. Si esta firma no es válida, no se puede confiar en ninguna clave de esa zona, por lo que toda la cadena debe considerarse inválida.
- **Verificación de la firma RRset DS utilizando la ZSK del padre:** El RRset DS se encuentra firmado por la ZSK del padre. Si esta verificación falla, el RRset DS tampoco es confiable, lo que impide autenticar el vínculo entre el padre y el hijo.

Ambas validaciones se realizan mediante la función `verify_rrsig`.

Verificación de la correspondencia entre el DS y la KSK de la zona hija

Si ambas firmas del paso anterior son válidas, el algoritmo procede a verificar que alguno de los registros DS obtenidos en la zona padre corresponde efectivamente con la clave KSK de la zona hija.

Para ello se itera por cada registro DS y se compara su `digest` con el valor calculado a partir de la concatenación del nombre canónico del apex de la zona hija y el RData del DNSKEY KSK de la zona hija, utilizando la función hash indicada en el campo `digest_type` del DS. Este calculo es el indicado en el RFC 4034 [2], sección 5.1.4.

Aclarar también que, si bien el campo `key tag` se utiliza tradicionalmente para ayudar a identificar una clave pública candidata de forma eficiente, el estándar advierte explícitamente que este valor **no garantiza unicidad**. Según el RFC 4034 [2], apéndice B, es posible que dos claves distintas produzcan el mismo `key tag`, dado que su cálculo corresponde a una función hash simplificada. Por lo tanto, este diseño no depende de dicho campo para determinar la correspondencia entre una firma y su DNSKEY, sino que valida directamente los `digest` contenidos en los registros DS, evitando así cualquier riesgo asociado a colisiones en el `key tag`.

Si algún DS coincide, se confirma que la zona padre efectivamente delega y certifica la clave KSK de la hija.

Ascenso recursivo hasta la raíz

Una vez validado el vínculo entre el padre y el hijo, se debe continuar verificando la cadena de autenticación hacia arriba. El algoritmo actúa así:

- Si la zona padre actual corresponde a la zona raíz, la verificación concluye exitosamente, ya que la raíz constituye el ancestro de confianza y no necesita validación adicional, además no se puede seguir subiendo ya que la zona raíz no tiene padre.
- En caso contrario:
 - La zona padre se convierte ahora en la zona hija del siguiente nivel
 - Se determina cuál es el nuevo nombre de la zona padre
 - Se extrae el `DnskeyRdata` correspondiente (que será utilizada en el próximo cálculo de DS)
 - El proceso completo se repite recursivamente sobre ese nivel superior.

Resultado final

La cadena DS se considera válida únicamente si todas las etapas se completan exitosamente:

- El DS se obtuvo correctamente desde la zona padre.
- El DNSKEY RRset y el DS RRset del padre fueron firmados y verificados correctamente.
- Algún registro DS coincide con el `digest` calculado a partir de la KSK de la zona hija.
- La verificación se mantiene válida en cada nivel superior hasta alcanzar la zona raíz.

Solo entonces la clave KSK de la zona inicial puede considerarse auténtica y correctamente anclada en la cadena de confianza de DNSSEC.

7.2.6. Ejemplo real de validación DNSSEC

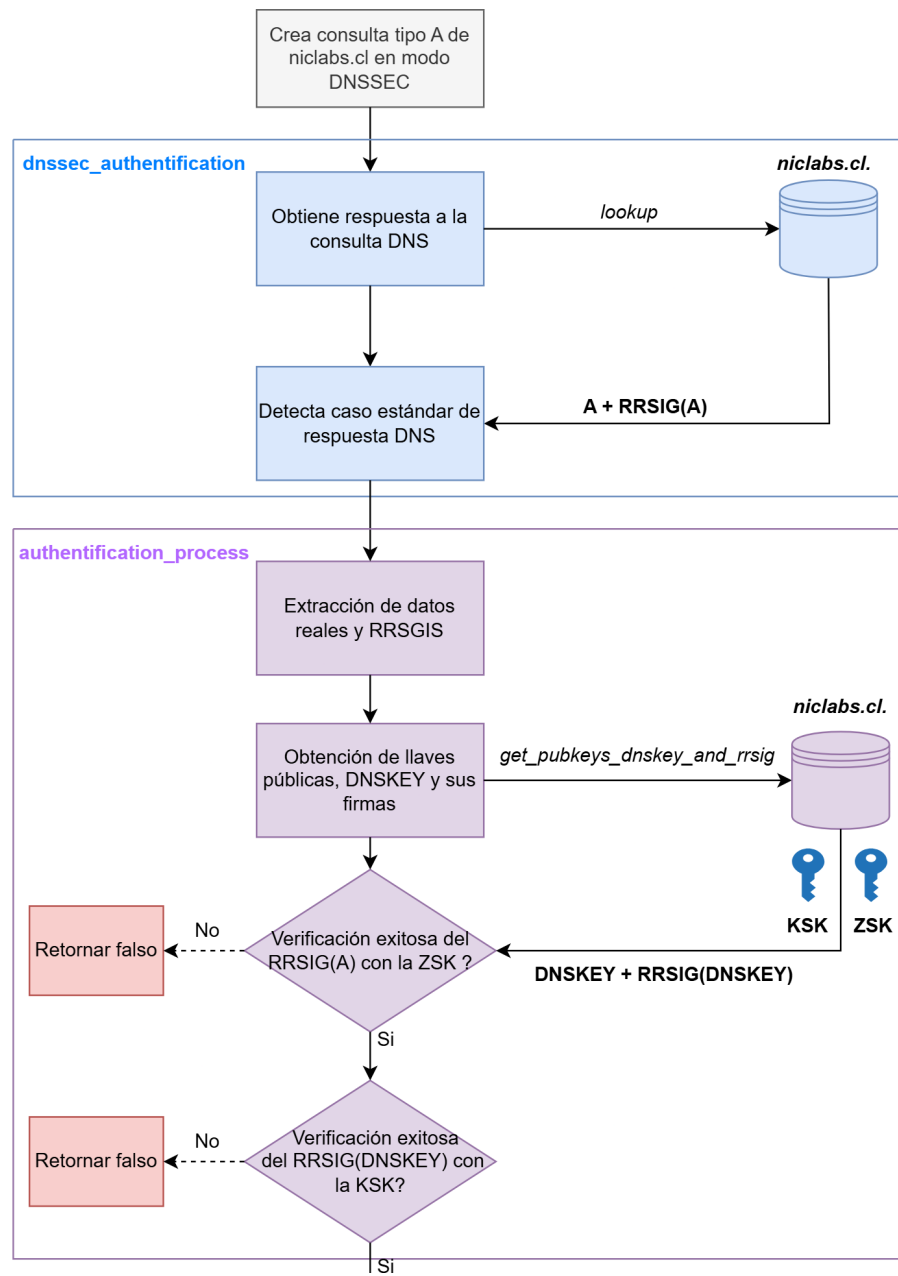
Para poder probar que todo lo anterior funciona, se debe crear una consulta DNSSEC válida y usar la función de autenticación para verificar la respuesta. En código sería:

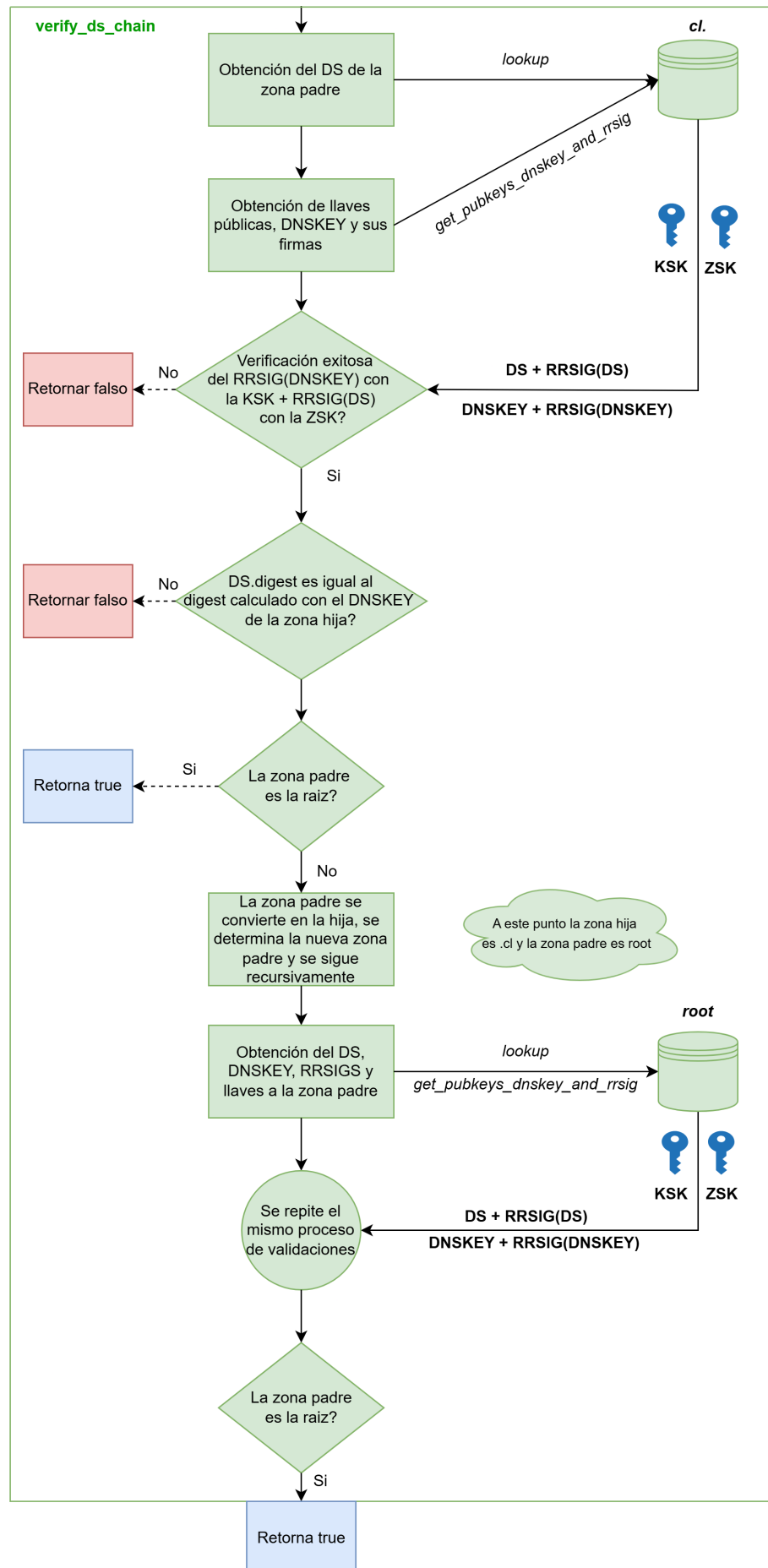
```

1  let dns_query = create_dns_opt_query("niclabs.cl", Rrtype::A);
2  let authentication = dnssec_authentication(&dns_query).await;
3  println!("the authentication is {:?}", authentication);

```

Este proceso retorna verdadero, confirmando que la respuesta DNSSEC es válida y que todas las firmas y la cadena de autenticación han sido verificadas correctamente. Pero además en consola se imprimen todos los pasos intermedios del proceso, mostrando como se obtienen las claves, los RRsets, las firmas y los resultados de cada verificación. Mostrar este output completo aquí no es práctico, ya que es demasiado extenso, por lo que a continuación se muestra un diagrama de flujo que ilustra visualmente el proceso de validación.





Como se observa en el diagrama, el proceso de validación DNSSEC es complejo y requiere múltiples pasos y verificaciones para garantizar la autenticidad e integridad de las respuestas DNS, específicamente requiere de muchos llamados a las distintas base de datos de las zonas involucradas. Aclarar que en el diagrama se pueden ver flechas punteadas, estas indican el camino alternativo en caso de que alguna verificación falle, lo que hace que todo el proceso retorne falso inmediatamente, pero en este ejemplo en concreto todas las verificaciones son exitosas y se llega al final del diagrama retornando verdadero.

También decir que este proceso sería muy similar al que se haría si se tuviera comunicación TCP/UDP entre servidor y resolver. Lo único distinto sería que en vez de obtener las respuestas con llamados a la función `lookup_data`, se haría enviando consultas DNS a través de la red, teniendo que además hacer parsing de las respuestas recibidas. Pero en términos de lógica de validación, sería idéntico.

Así como este proceso retorna verdadero para una consulta de tipo A válida, también se probó con todas las siguientes consultas:

Consulta con resultado NXDOMAIN

```
1   let dns_query = create_dns_opt_query("unknown.niclabs.cl", Rrtype
    ::A);
2   let authentication = dnssec_authentication(&dns_query).await;
```

Este caso sería el que usa la función `nxdomain_authentication_process` y retorna verdadero, confirmando que la respuesta NXDOMAIN es válida bajo DNSSEC.

Consulta con resultado NO_DATA que no es un delegation point

```
1   let dns_query = create_dns_opt_query("www.example.com", Rrtype::
    TXT);
2   let authentication = dnssec_authentication(&dns_query).await;
```

Este caso sería el que usa la función `nodata_authentication_process` y retorna verdadero, confirmando que la respuesta NO_DATA es válida bajo DNSSEC.

Consulta con resultado NO_DATA que es un delegation point

```
1   let dns_query = create_dns_opt_query("b.example.com", Rrtype::A);
2   let authentication = dnssec_authentication(&dns_query).await;
```

Este caso también usa la función `nodata_authentication_process` pero en este caso se añade el DS asociado a la delegación. Retorna verdadero, confirmando que la respuesta NO_DATA es válida bajo DNSSEC.

Consulta de tipo DNSKEY

```
1 let dns_query = create_dns_opt_query("niclabs.cl", Rrtype::DNSKEY);
2 let authentication = dnssec_authentication(&dns_query).await;
```

Este caso activa el modo especial de autenticación para consultas DNSKEY en la función `authentication_process`, usando la KSK para verificar la firma del RRset DNSKEY. Retorna verdadero, confirmando que la respuesta DNSKEY es válida bajo DNSSEC.

Consulta generada por wildcard

```
1 let dns_query = create_dns_opt_query("unknown.example.com",
Rrtype::A);
2 let authentication = dnssec_authentication(&dns_query).await;
```

En este caso hay que quitar el comentario del registro wildcard en el masterfile de la zona `example.com` para que funcione. Este caso usa la función `wildcard_authentication_process` y retorna verdadero, confirmando que la respuesta generada por el wildcard es válida bajo DNSSEC.

Consulta que utiliza un CNAME

```
1 let dns_query = create_dns_opt_query("blog.example.com", Rrtype::
A);
2 let authentication = dnssec_authentication(&dns_query).await;
```

Este caso ocupa la función estándar `authentication_process`, pero lo distinto es que contiene dos respuestas de dos tipos distintos: un CNAME y un A. Ambos registros tienen su respectivo RRSIG y ambos son validados correctamente. Retorna verdadero, confirmando que la respuesta con CNAME es válida bajo DNSSEC.

Capítulo 8

Discusiones y Resultados

8.1. Uso del lenguaje Rust y sus desventajas

La librería `dns-rust` está escrita en Rust, por lo que el desarrollo del proyecto se realizó íntegramente en este lenguaje. Rust es un lenguaje de programación de sistemas que destaca por su seguridad de memoria, su modelo de concurrencia libre de condiciones de carrera y su alto rendimiento sin necesidad de un recolector de basura. Estas características lo convierten en una opción atractiva para la implementación de software de bajo nivel, como servidores de nombres DNS.

No obstante, Rust también presenta ciertos desafíos importantes que se hicieron evidentes durante el desarrollo. En particular, el modelo de *ownership* y *borrowing* (uno de sus pilares seguridad) introduce restricciones estrictas sobre cómo se gestionan las referencias y la mutabilidad. Si bien estas reglas previenen errores comunes como *use-after-free* o accesos concurrentes no seguros, también complican significativamente el diseño de estructuras de datos complejas.

Este problema se manifestó de forma especialmente clara al intentar implementar una estructura jerárquica de zonas DNS basada en árboles, tal como sugieren los estándares. La necesidad de mantener referencias mutables hacia nodos padres e hijos, junto con los largos ciclos de vida asociados a estas referencias, generó importantes dificultades para cumplir simultáneamente con las garantías del compilador. Debido a estas limitaciones, la implementación final debió abandonar el enfoque basado en árboles y optar por una estructura alternativa compuesta por *hash maps* anidados, que resulta más compatible con el modelo de propiedad de Rust y evita los problemas asociados a referencias cíclicas.

8.2. Uso de mapas anidados en lugar de árboles

Como se mencionó en la sección anterior, la implementación inicial del servidor de nombres DNS buscaba modelar la jerarquía natural de las zonas mediante una estructura de árbol,

donde cada nodo representaba un `RRSet` y contenía referencias hacia sus nodos descendientes. Sin embargo, debido a las limitaciones impuestas por el sistema, se optó finalmente por reemplazar esta estructura por una basada en mapas (`HashMap`) anidados.

Esto plantea la pregunta: ¿Afecta este cambio el rendimiento del servidor?

En términos de eficiencia, la respuesta breve es que los mapas anidados suelen ser más rápidos para las operaciones típicas de un servidor DNS. Dado un nombre de dominio y un tipo de registro, acceder al `RRSet` correspondiente es una operación de complejidad $O(1)$ en el caso de mapas anidados, mientras que en una estructura de árbol sería $O(h)$, donde h corresponde a la altura del árbol. En otras palabras, los mapas ofrecen tiempos de acceso constantes, mientras que los árboles implican recorrer la jerarquía nodo por nodo.

La verdadera desventaja de abandonar una estructura arbórea es la pérdida de la representación explícita de la jerarquía de dominios. Esta organización jerárquica podría facilitar ciertas operaciones conceptualmente dependientes de la estructura DNS (por ejemplo, recorridos ascendentes o búsquedas basadas en prefijos de dominio). Sin embargo, en la práctica, y considerando el conjunto de RFC analizados durante este proyecto, ninguna de las operaciones implementadas requirió dicha jerarquía explícita. No obstante, no puede descartarse que futuras extensiones o características avanzadas de un servidor DNS puedan beneficiarse de una estructura de árbol.

Es importante destacar, además, que el uso de una estructura jerárquica es una recomendación de los estándares (una sugerencia (*SHOULD*)) y no un requisito obligatorio. Por lo tanto, la decisión de utilizar mapas anidados en lugar de árboles no compromete el cumplimiento de las especificaciones, y ofrece una alternativa más eficiente y más compatible con las restricciones del lenguaje Rust.

8.3. Limitaciones y errores encontrados en la librería `dns-rust`

Como se vió a lo largo del desarrollo del proyecto, la librería `dns-rust` presenta varias limitaciones y errores que afectan la implementación de un servidor de nombres DNS completo basado en los estándares. A continuación, se detallan algunos de los problemas más relevantes encontrados:

8.3.1. Funciones de parsing

En DNS, los mensajes se transmiten mediante los protocolos UDP o TCP utilizando un formato en bytes. Esto implica que debe existir un mecanismo para transformar esos bytes en estructuras de datos utilizables (`from_bytes`) y, en sentido inverso, convertir dichas estructuras nuevamente a su representación binaria (`to_bytes`).

El principal problema surge porque estas funciones implementan compresión de nombres

de dominio, una técnica estándar en DNS que permite representar nombres repetidos de manera más eficiente dentro de un mismo mensaje. Sin embargo, esta compresión también introduce complejidad adicional en el proceso de *parsing*. En particular, cuando un mensaje contiene al menos un **Resource Record** en cualquiera de sus secciones (**Answer**, **Authority** o **Additional**), la función `to_bytes` intenta aplicar compresión a los nombres incluidos dentro del campo `Rdata`. El problema es que dicha compresión está implementada incorrectamente, lo que provoca que, al intentar reconstruir luego el mensaje mediante `from_bytes`, el proceso falle con un error.

Se intentó corregir este comportamiento modificando directamente el código fuente de la librería; sin embargo, debido a la gran extensión del código, la ausencia de documentación y la complejidad de su estructura interna, no fue posible localizar ni resolver la causa exacta del problema. También se exploró la posibilidad de implementar versiones alternativas de `from_bytes` y `to_bytes` sin compresión, pero estos intentos tampoco tuvieron éxito.

8.3.2. Falta de implementación de `to_canonical_bytes` para los `Rdata`

Otra limitación relevante es que ninguno de los tipos de `Rdata` incluidos en la librería implementa la función `to_canonical_bytes`, la cual es fundamental para la validación DNS-SEC. Esta función debe producir la representación canónica en bytes del `Rdata`, siguiendo estrictamente las reglas de canonicalización definidas en los RFC, y es indispensable para el cálculo y la verificación de las firmas digitales (RRSIG).

Debido a esta ausencia, fue necesario implementar manualmente `to_canonical_bytes` para cada tipo de `Rdata` usado en el proyecto, basándose directamente en las especificaciones formales de los RFC correspondientes y asegurando que cada uno cumpliera con el formato requerido por DNSSEC.

8.3.3. Bug en la función de inserción en la sección `Authority`

La función `add_authorities` está diseñada para incorporar registros en la sección `Authority` de un mensaje DNS. Sin embargo, su implementación presenta un error crítico: aunque la función obtiene correctamente los registros de la sección `Authority` y actualiza el contador correspondiente, en la última línea invoca `set_answer` en lugar de `set_authority`.

```
1  pub fn add_authorities(&mut self, mut authorities: Vec<
   ResourceRecord>) {
2      let mut msg_authorities = self.get_authority();
3
4      msg_authorities.append(&mut authorities);
5      self.header.set_nscount(msg_authorities.len() as u16);
6      self.set_answer(msg_authorities);
7  }
```

Como consecuencia, todos los `ResourceRecord` que deberían ser añadidos a `Authority` terminan siendo escritos erróneamente en la sección `Answer`. Este comportamiento altera la estructura del mensaje DNS y genera inconsistencias graves, especialmente en contextos donde la sección `Authority` es fundamental, como en respuestas delegadas o en validaciones DNSSEC.

Para corregir el problema, basta con reemplazar la llamada final por `set_authority`, lo que garantiza que los registros se almacenen en la sección adecuada.

Este bug es especialmente severo, ya que afecta a cualquier implementación que emplee esta función para construir mensajes DNS y podría provocar respuestas incorrectas o inválidas. Resulta llamativo que un error de esta magnitud haya pasado desapercibido en la librería original.

8.4. Testing del proyecto

Para probar el funcionamiento de este proyecto, primero es necesario clonar el repositorio `name-server-rust` [16] desde GitHub.

Dentro de este repositorio se encuentra el archivo `main.rs`, el cual reúne múltiples casos de uso del servidor: pruebas completamente locales (como visualización de las bases de datos internas, los `RRSet DS` y el catálogo) y pruebas cliente-servidor, permitiendo inicializar el servidor en modo UDP o en un modo híbrido (UDP y TCP simultáneamente).

Además, en la carpeta `bin` se incluye un archivo `client.rs`, que implementa un cliente DNS simple capaz de enviar consultas al servidor para facilitar las pruebas.

A continuación se describen los principales casos de uso disponibles para testear el sistema:

8.4.1. Servidor UDP con soporte para consultas paralelas

Una de las características más relevantes del servidor es su capacidad para manejar múltiples consultas concurrentes. Este comportamiento puede verificarse ejecutando el servidor en modo UDP y enviando varias consultas en paralelo desde el cliente, utilizando tareas asincrónicas provistas por la librería `tokio`.

Para probar este caso de uso:

1) Inicie el servidor en modo UDP ejecutando en una terminal:

```
1 cargo run --bin nameserver-rust -- 1
```

2) Envíe consultas paralelas desde el cliente ejecutando en otra terminal:

```
1 cargo run --bin client -- 1
```

Este comando hace que el cliente envíe dos consultas DNS en paralelo.

Al observar la salida de la consola del servidor, podrá ver que ambas consultas son recibidas y procesadas de forma concurrente. Cada una se atiende de manera independiente, generando finalmente dos respuestas DNS, lo que demuestra la capacidad del servidor para manejar múltiples solicitudes simultáneas sin bloquear su ejecución.

8.4.2. Servidor híbrido: manejo de truncamiento (TC) y consultas paralelas en TCP

El proyecto también permite ejecutar el servidor en un modo híbrido, en el cual se levantan simultáneamente dos instancias del servicio DNS: una escuchando consultas mediante UDP y otra mediante TCP.

1) Ejecute el servidor en modo híbrido con :

```
1 cargo run --bin nameserver-rust -- 2
```

Este comando inicia dos listeners en paralelo:

- Un servidor UDP, encargado de manejar consultas rápidas y de pequeño tamaño
- Un servidor TCP, necesario para consultas grandes o cuando se requiere confiabilidad adicional.

2) Envíe una consulta grande desde el cliente con:

```
1 cargo run --bin client -- 2
```

En este caso de uso, el cliente primero envía una consulta DNS cuyo tamaño supera los 512 bytes, que es el máximo permitido por defecto para recibir mensajes UDP en el servidor (es decir, el UDP payload size configurado por el propio servidor). Debido a que la consulta excede este límite, la instancia UDP del servidor no puede procesarla completamente y responde con un mensaje marcado con el bit TC (Truncated) activado.

Al recibir esta respuesta trunca, el cliente detecta correctamente la condición TC y vuelve a enviar la misma consulta, pero ahora utilizando TCP, tal como exige la especificación DNS para manejar mensajes que no caben en un paquete UDP.

Para demostrar que la implementación TCP también soporta concurrencia, el cliente no solo reenvía la consulta por TCP, sino que clona la consulta y envía dos solicitudes simultáneas. Ambas llegan al servidor TCP, el cual las procesa paralelamente y retorna las respuestas correspondientes.

Este flujo permite verificar tres características clave del servidor híbrido:

- La correcta señalización del bit TC ante respuestas demasiado grandes en UDP.
- La capacidad del cliente para detectar respuestas truncadas y reenviar consultas por TCP.
- La capacidad del servidor TCP para manejar múltiples consultas concurrentes sin bloquearse.

8.4.3. Manejo de consultas mal formadas (FORMERR)

Este caso de uso permite evaluar la capacidad del servidor para identificar consultas DNS con errores de formato. En el caso 3 del servidor, este se ejecuta en un modo que espera una única consulta UDP con el propósito de verificar el manejo de errores según lo definido por el estándar DNS.

Cuando el servidor recibe una consulta, el proceso de validación no lo realiza directamente el código del servidor, sino la función `from_bytes` provista por la librería `dns-rust`, la encargada de transformar una secuencia de bytes en una estructura interna `DnsMessage`. Es precisamente esta función la que determina si la consulta cumple la sintaxis del protocolo DNS. Si detecta una violación (por ejemplo, una etiqueta cuyo tamaño excede los 63 bytes permitidos) la función devuelve un error. Ante este escenario, el servidor construye una respuesta válida a nivel de encabezado e incluye el código de retorno `RCODE = FORMERR`, indicando un error de formato en la consulta original.

Es importante aclarar que este mecanismo depende completamente del correcto funcionamiento de `from_bytes`. Existen otros tipos de consultas mal formadas que, de acuerdo con los RFC, también deberían producir un `FORMERR`, pero que la implementación actual de dicha función no detecta correctamente. En esos casos, el servidor no puede señalar el error a pesar de que la consulta sea inválida. Este punto es relevante para interpretar los resultados de este caso de prueba y comprender las limitaciones actuales de la librería utilizada.

Cliente enviando una consulta DNS mal formada

En el caso 3 del cliente, se construye explícitamente una consulta DNS con un encabezado válido pero con un error crítico en el nombre de dominio: la primera etiqueta declara una longitud de 64 bytes, superando el máximo permitido de 63. Esta inconsistencia constituye un error de formato según el RFC 1035 [15], por lo que el servidor responde con `FORMERR`.

El cliente envía esta consulta al servidor mediante UDP y, al recibir la respuesta, reconstruye su encabezado para verificar que el código de error incluido sea efectivamente **FORMERR**. Esto permite constatar que el servidor (a través de **from_bytes**) identificó correctamente este caso puntual como una violación sintáctica del protocolo.

8.4.4. Manejo de consultas DNSSEC en UDP y TCP

Cuando el cliente opera en los modos 4 (UDP) y 5 (TCP), construye consultas DNS compatibles con DNSSEC utilizando EDNS(0) mediante la inclusión de un registro OPT en la sección **Additional**. Este registro OPT es lo que permite extender el protocolo DNS clásico e indicar explícitamente que el cliente quiere recibir datos DNSSEC.

Cómo el cliente indica que quiere DNSSEC

El cliente agrega un registro OPT con tres elementos clave:

- **UDP Payload Size ampliado (ej. 4096 bytes):** Permite que las respuestas DNSSEC (que incluyen firmas criptográficas y por lo tanto son más grandes) puedan transmitirse sin truncamiento innecesario.
- **Versión EDNS = 0:** Es la versión estándar requerida para que el servidor sepa que el mensaje usa EDNS(0).
- **DO bit activado (DNSSEC OK):** Le dice explícitamente al servidor que el cliente quiere una respuesta con información DNSSEC, es decir, **RRsets** firmados, con sus registros **RRSIG** correspondientes.

El cliente construye exactamente la misma consulta tanto para UDP como para TCP; la única diferencia es el transporte.

Como el server maneja las consultas DNSSEC

Cuando el servidor recibe una consulta (en el modo híbrido, que se instancia con el caso numero 2) que incluye un registro OPT con el bit DO activado, interpreta que el cliente está solicitando datos DNSSEC. Así que por lo tanto responde incluyendo los registros **RRSIG** correspondientes junto con los **RRSet** solicitados.

8.4.5. Verificación local de estructuras de datos y RRsets DNSSEC

Para asegurar la correcta construcción y consistencia de las estructuras internas del servidor DNS, se implementaron varios casos de prueba cuyo propósito es inspeccionar y validar visualmente el contenido de los componentes críticos del sistema. Cada caso imprime en

consola el estado actual de las estructuras relevantes, permitiendo confirmar su correcta inicialización antes de utilizarlas en las respuestas del servidor o incluirlas en los `master files`.

Revisión del Catálogo

El catálogo mantiene referencias a todas las zonas administradas por el servidor. En estas pruebas se extrae la estructura interna asociada a una zona específica (por ejemplo, `niclabs.cl`) y se imprime su contenido completo. Esto permite verificar:

- Los nombres de dominio registrados.
- Los tipos de `RRsets` presentes en cada dominio.
- El contenido de cada `Rdata` almacenado.

Esta revisión asegura que la carga de zonas y la organización general del catálogo son correctas.

Revisión de todas las estructuras de datos de zonas

El servidor mantiene múltiples estructuras internas que representan zonas completas: `DATA_STRUCTURE_ROOT`, `DATA_STRUCTURE_COM`, `DATA_STRUCTURE_CL`, `DATA_STRUCTURE_NICLABS`, `DATA_STRUCTURE_EXAMPLE`.

Cada una puede imprimirse íntegramente en consola para validar:

- Los `RRsets` almacenados y sus owner names.
- La presencia y consistencia de los registros `RRSIG`.
- La correcta estructura interna de cada zona.
- La coherencia entre zonas relacionadas (por ejemplo, `com` y `example.com`).

Estas impresiones funcionan como una auditoría detallada de la información autoritativa mantenida por cada zona.

Verificación de los DS generados

Como parte del proceso DNSSEC, se implementaron funciones para construir los registros DS de las zonas hijas. Los casos de prueba imprimen los DS generados para:

- `example.com`.

- `com.`
- `cl.`
- `niclabs.cl.`

La impresión de estos registros es fundamental, ya que deben copiarse manualmente e insertarse posteriormente en los **master files** de las zonas superiores.

Verificación de claves DNSSEC

Otro caso imprime las claves almacenadas en `GLOBAL_ZONE_KEY_STORE`. Esto permite revisar:

- Las claves ZSK y KSK asociadas a cada zona.
- Que cada conjunto de claves está correctamente vinculado a su zona correspondiente.

Estas verificaciones proporcionan una validación final del estado interno del sistema antes de realizar pruebas más complejas en el servidor.

8.4.6. Casos de autenticación criptográfica DNSSEC

Del caso 16 al caso 22 en el archivo `main.rs` es posible ejecutar distintas autenticaciones criptográficas DNSSEC de manera local utilizando la función `dnssec_authentication`. Estos casos corresponden exactamente a los mismos escenarios analizados previamente en la sección 7.2.6.

Al ejecutar cualquiera de estos casos, la consola mostrará el detalle completo de cada etapa del proceso de verificación DNSSEC, incluyendo la validación de firmas, la comprobación de los `RRSIG`, la verificación de los `DS`, y la reconstrucción de la cadena de confianza, hasta llegar al resultado final.

Si la autenticación se realiza correctamente, siempre se imprimirá el siguiente mensaje al final del proceso:

```
1 Reached root full chain verified.
2 the DS chain was verify with : true
3 the authentication is true
```

Este mensaje confirma que toda la cadena de validación, desde la zona consultada hasta la raíz, fue verificada con éxito.

Capítulo 9

Conclusiones

El objetivo general de este proyecto de memoria fue desarrollar un servidor de nombres DNS iterativo en el lenguaje Rust, utilizando la librería `dns-rust`, y que cumpliera rigurosamente con el estándar de DNSSEC, siguiendo las especificaciones establecidas en los RFC 1034 [14], 1035 [15], 4033 [1], 4034 [2] y 4035 [3]. Se puede afirmar que dicho objetivo fue alcanzado de manera íntegra. El resultado final consistió en un proyecto de más de 3000 líneas de código puro, capaz de manejar consultas simultáneas tanto en UDP como en TCP, con soporte completo para EDNS0 y DNSSEC, y con la capacidad de gestionar múltiples zonas para emular adecuadamente la cadena de autenticación requerida por el modelo de seguridad DNSSEC.

Asimismo, se diseñó e implementó un algoritmo de búsqueda que responde de manera correcta y consistente para cada uno de los casos evaluados, siguiendo las reglas definidas por los estándares. El sistema incorpora firmas criptográficas reales que fueron generadas, insertadas y validadas correctamente durante el proceso de verificación. Además, cuenta con manejo explícito de errores, detección de loops y un desempeño estable incluso al procesar múltiples consultas en paralelo.

Durante el desarrollo también se obtuvieron conclusiones relevantes respecto a la estructura interna de los datos. En particular, se determinó que no es estrictamente necesario utilizar la estructura jerárquica de árbol sugerida por los RFC, y que incluso si hubiese sido deseable implementarla, Rust dificulta considerablemente este enfoque debido a su modelo de propiedad y los problemas que surgen al intentar crear estructuras de árboles mutables y compartidas. En consecuencia, se optó por un enfoque distinto, más adecuado para el lenguaje, que permitió avanzar sin comprometer el rendimiento ni la corrección del sistema.

Además, durante la implementación se identificaron varios problemas en la librería `dns-rust`, la cual se publicita como una librería lista y funcional. En concreto, se descubrieron dos bugs importantes: uno en las funciones de parsing y otro asociado al manejo de la sección `Additional`. También se evidenció la ausencia de funcionalidades fundamentales para un sistema DNSSEC, como la conversión de los distintos tipos `Rdata` a sus representaciones canónicas, lo que obligó a extender la librería para cumplir con los requisitos del protocolo.

A pesar de los avances, aún queda trabajo por realizar. No se logró implementar un sistema

formal de testing utilizando herramientas especializadas, por lo que el proyecto actualmente carece de tests unitarios y de integración que permitan garantizar más exhaustivamente su correcto funcionamiento. Adicionalmente, el servidor está todavía lejos de ser una solución preparada para entornos reales, ya que faltan numerosos RFC por implementar, mecanismos de comunicación real con servidores externos, herramientas de seguridad como firewalls DNS para mitigar ataques, entre muchas otras funcionalidades necesarias para una operación a nivel productivo.

Sin embargo, a pesar de estas limitaciones, el proyecto constituye una base sólida y extensible para continuar el desarrollo de un servidor DNS iterativo con soporte DNSSEC completamente funcional. Se espera que esta implementación pueda servir como punto de partida para que el laboratorio **NIC Labs** continúe profundizando y expandiendo este trabajo en el futuro, si así lo estima conveniente.

Bibliografía

- [1] Arends, Roy, Austein, Rob, Larson, Matt, Massey, Dan, and Rose, Scott. Rfc 4033: Dns security introduction and requirements. *RFC Editor*, 2005. URL: <https://www.rfc-editor.org/rfc/pdf/rfc4033.txt.pdf>.
- [2] Arends, Roy, Austein, Rob, Larson, Matt, Massey, Dan, and Rose, Scott. Rfc 4034: Resource records for the dns security extensions. *RFC Editor*, 2005. URL: <https://www.rfc-editor.org/rfc/pdf/rfc4034.txt.pdf>.
- [3] Arends, Roy, Austein, Rob, Larson, Matt, Massey, Dan, and Rose, Scott. Rfc 4035: Protocol modifications for the dns security extensions. *RFC Editor*, 2005. URL: <https://www.rfc-editor.org/rfc/pdf/rfc4035.txt.pdf>.
- [4] Bastián Corrales. Resúmenes de rfc's (notion). *Sitio web*, s.f. URL: <https://www.rfc-editor.org/rfc/pdf/rfc1034.txt.pdf>.
- [5] GitHub Security Advisories. Ghsa-37wc-h8xc-5hc4: Hickory dns bug. *Sitio web*, 2023. URL: <https://github.com/advisories/GHSA-37wc-h8xc-5hc4>.
- [6] Hickory Project. Hickory dns. *Sitio web*, s.f. URL: <https://github.com/hickory-dns/hickory-dns>.
- [7] IETF. Birthday attacks en dns. *RFC 5452*, 2009. URL: <https://datatracker.ietf.org/doc/html/rfc5452#section-5>.
- [8] IETF. Ataques de ciberseguridad por validación incorrecta de nombres de dominio. *RFC 9267*, 2022. URL: <https://www.rfc-editor.org/rfc/rfc9267.html#name-label-and-name-length-valid>.
- [9] InfoWorld. Why rust is a memory-safe language. *InfoWorld*, 2024. URL: <https://www.infoworld.com/article/2336661/rust-memory-safety-explained.html>.
- [10] Internet Systems Consortium. Bind 9. *Sitio web*, s.f. URL: <https://www.isc.org/bind/>.
- [11] Kelley, Simon. Dnsmasq documentation. *Sitio web*, s.f. URL: <https://thekelleys.org.uk/dnsmasq/doc.html>.
- [12] MITRE. Cve-2020-25110: Vulnerabilidad en ethernut. *CVE Database*, 2020. URL: <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2020-25110>.

- [13] MITRE. Cve-2020-15795: Vulnerabilidades en iot por labels mal validados. *CVE Database*, 2023. URL: <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2020-15795>.
- [14] Mockapetris, Paul. Rfc 1034: Domain names - concepts and facilities. *RFC Editor*, 1987. URL: <https://www.rfc-editor.org/rfc/pdf/rfc1034.txt.pdf>.
- [15] Mockapetris, Paul. Rfc 1035: Domain names - implementation and specification. *RFC Editor*, 1987. URL: <https://www.rfc-editor.org/rfc/pdf/rfc1035.txt.pdf>.
- [16] NICLabs. nameserver-rust: Dns server implementation in rust. *GitHub Repository*, 2025. URL: <https://github.com/niclabs/nameserver-rust>.
- [17] NLnet Labs. Issue #520: Bug en domain crate. *GitHub Issues*, 2025. URL: <https://github.com/NLnetLabs/domain/issues/520>.
- [18] NLnet Labs. Domain crate. *Sitio web*, s.f. URL: <https://github.com/NLnetLabs/domain/tree/main/>.
- [19] PowerDNS. Powerdns. *Sitio web*, s.f. URL: <https://www.powerdns.com/>.
- [20] Schwartz, Michael, Vixie, Paul, Alexander, Michael, Floyd, Sally, Kraft, Walter, and Schulzrinne, Henning. Rfc 6891: Extension mechanisms for dns (edns(0)) — updated specification. *RFC Editor*, 2013. URL: <https://www.rfc-editor.org/rfc/pdf/rfc6891.txt.pdf>.
- [21] Thomassen, Martin. Towards an industry best practice for ds automation. *ICANN 84 Tech Day*, 2025. URL: https://static.sched.com/hosted_files/icann84/e0/2.%202025-10-27%20ICANN%2084%20Tech%20Day%2C%20Towards%20an%20Industry%20Best%20Practice%20for%20DS%20Automation%20-Thomassen.pdf.
- [22] Tokio Project. Tokio runtime. *Sitio web*, s.f. URL: <https://tokio.rs/>.
- [23] Vixie, Paul, Edwards, K, Beresford, P, and Chester, P. Rfc 2671: Extension mechanisms for dns (edns0). *RFC Editor*, 1999. URL: <https://www.rfc-editor.org/rfc/pdf/rfc2671.txt.pdf>.
- [24] Zero Day Initiative. Cve-2020-8625: Rce en bind. *Sitio web*, 2021. URL: <https://www.zerodayinitiative.com/blog/2021/2/24/cve-2020-8625-a-fifteen-year-old-rce-bug-returns-in-isc-bind-server>.